

**Napredni ADAS
algoritmi zasnovani
na obradi slike**



Sadržaj

- Pogled odozgor na vozilo i prostor oko njega (*Surround view*)
- Detekcija vozne trake i upozorenje o napuštanju vozne trake
- Detekcija objekata (pješaka, prometnih znakova, vozila)
- Nadzor vozača (*Driver monitoring*)
- Kamera kao zamjena za zrcalo (*Camera mirror replacement*)
- Detekcija parkirnog mjesta i pomoć pri parkiranju

**Pogled odozgor na vozilo i
prostor oko njega
(*Surround view*)**

Surround view algoritam

- Često se naziva i *around view* ili *surround view monitoring system*
- Stvara 2D sliku prostora oko vozila iz pogleda odozgor
- Pogled odozgor prema dolje 360° oko vozila
- Pomaže vozaču pri parkiranju vozila



Surround view algoritam

- Obično se sustav sastoji od 4-6 široko-kutnih (*fish-eye*) kamera montiranih na svim stranama vozila
 - Što su to *fish-eye* kamere? Koja je njihova karakteristika?
 - Zašto je ovdje potrebno koristiti *fish-eye* kamere?
 - Kada je potrebno koristiti 6 kamera?
- Slaže se složeni prikaz okoline vozila i prikazuje vozaču u stvarnom vremenu za vrijeme parkiranja



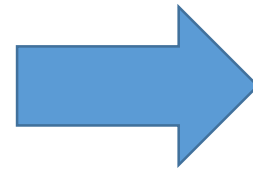
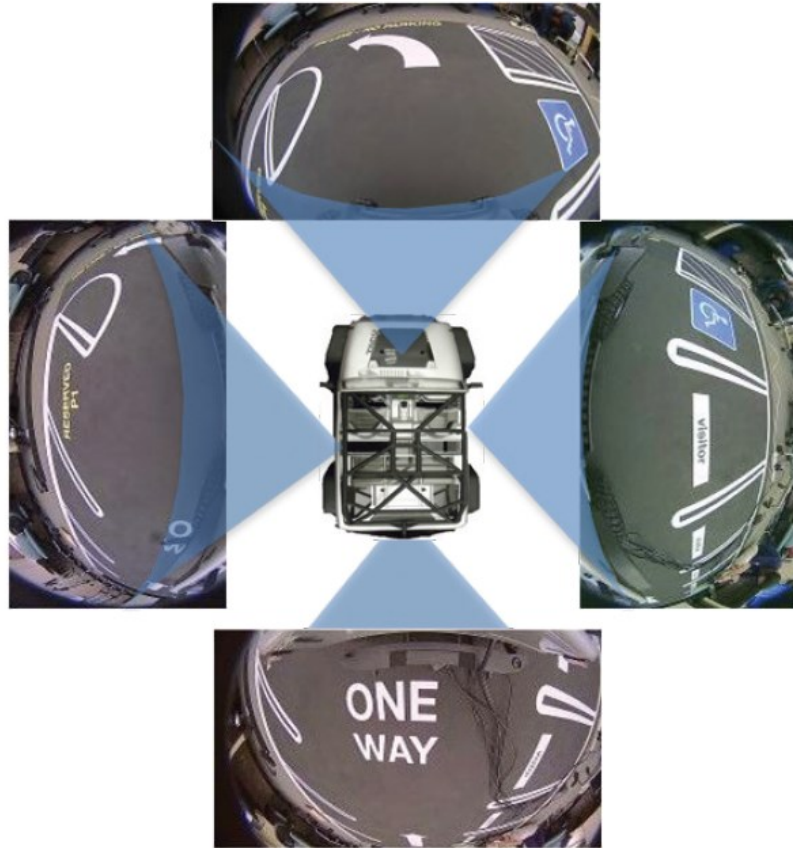
Surround view algoritam

- Primjer razrađen u nastavku → *Texas Instruments (TI) 360-degree surround view* za ADAS SoC procesore TDA2x (ALPHA ploča) [1],[2]
- Kako izgledaju ulazni podaci za *surround view* algoritam?
 - Ilustracija *surround view* sustava kamera i *fish-eye* slika dobivenih s pojedine kamere



Surround view algoritam

- A što je potrebno dobiti kao izlaz *surround view* algoritma?



Kako generirati *surround view*?

- Osnovno rješenje sastoji se od dvije glavne algoritamske komponente
 - **Geometrijsko poravnanje**
 - Ispravlja distorziju u ulaznim okvirima koju unosi *fish-eye* leća i pretvara ih u 2D pogled iz ptičje perspektive
 - **Sinteza složenog pogleda**
 - Generira složeni *surround view* (2D pogled odozgor) nakon geometrijskih korekcija

Kako generirati *surround view*?

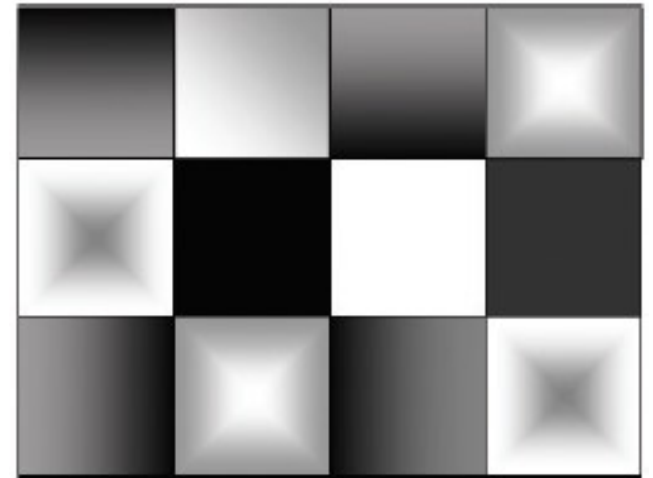
- Međutim, za stvaranje besprijekornog zajedničkog složenog pogleda odozgor, potreban je dodatni jedan ključni algoritam
 - **Fotometrijsko poravnanje**
 - Ispravlja nepodudaranje u svjetlini i boji između ulaznih slika sa susjednih kamera



Kako generirati *surround view*?

Geometrijsko poravnanje

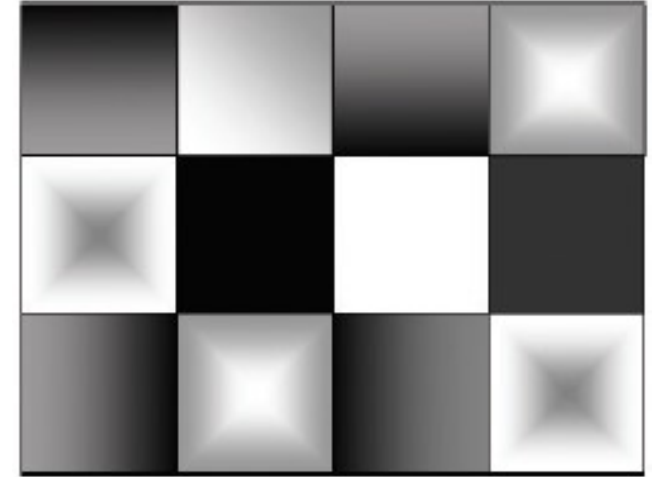
- Često se naziva i kalibracija
- Nužna komponenta *surround view* sustava
- Uključuje
 - Korekciju izobličenja *fish-eye* leće (kalibraciju)
 - Transformaciju perspektive



Kako generirati *surround view*?

Geometrijsko poravnanje

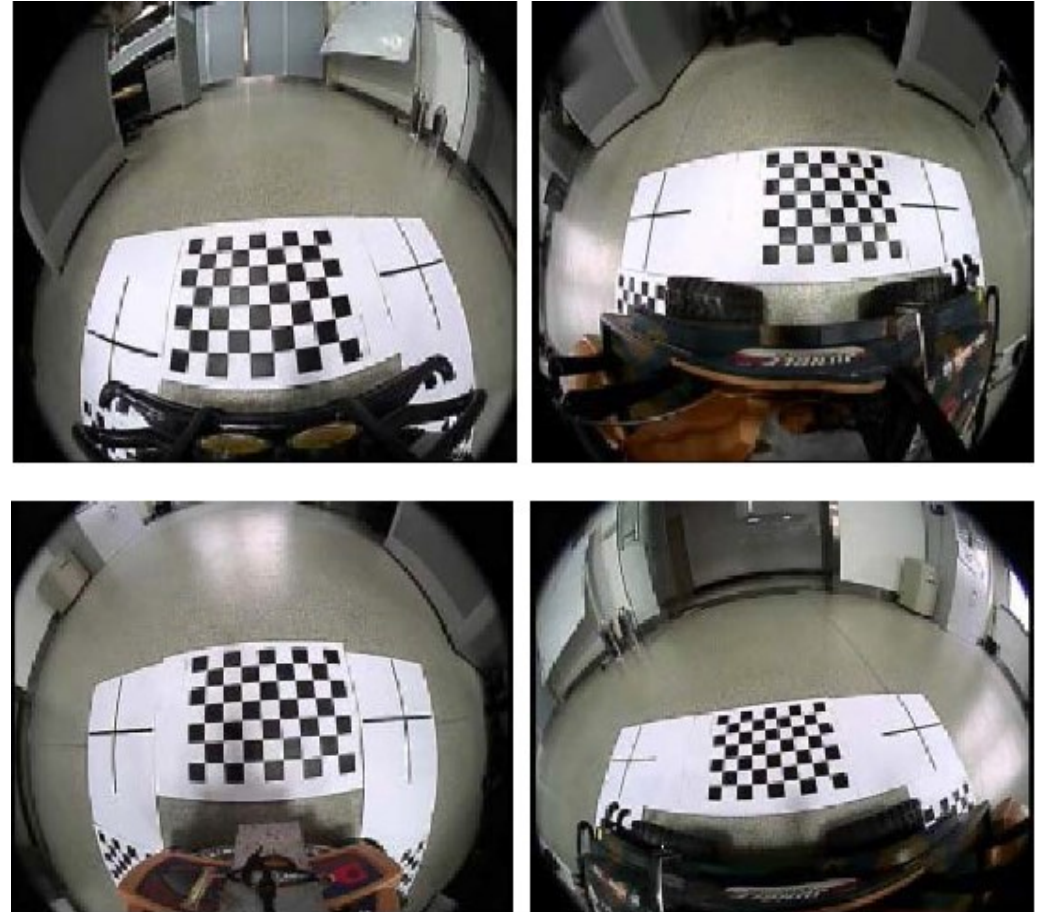
- Kalibracija zasnovana na uzorcima
 - 4 uzorka postavljena oko vozila
 - Npr. svaki uzorak smješten tako da se nalazi u vidnom polju dviju susjednih kamera
 - Primjena *lens distortion correction* (LDC) na svaki od 4 ulazna okvira



Kako generirati *surround view*?

Geometrijsko poravnanje

- Kalibracija zasnovana na uzorcima
- Kalibracijske slike oko automobila s kamera



Kako generirati *surround view*?

Geometrijsko poravnanje

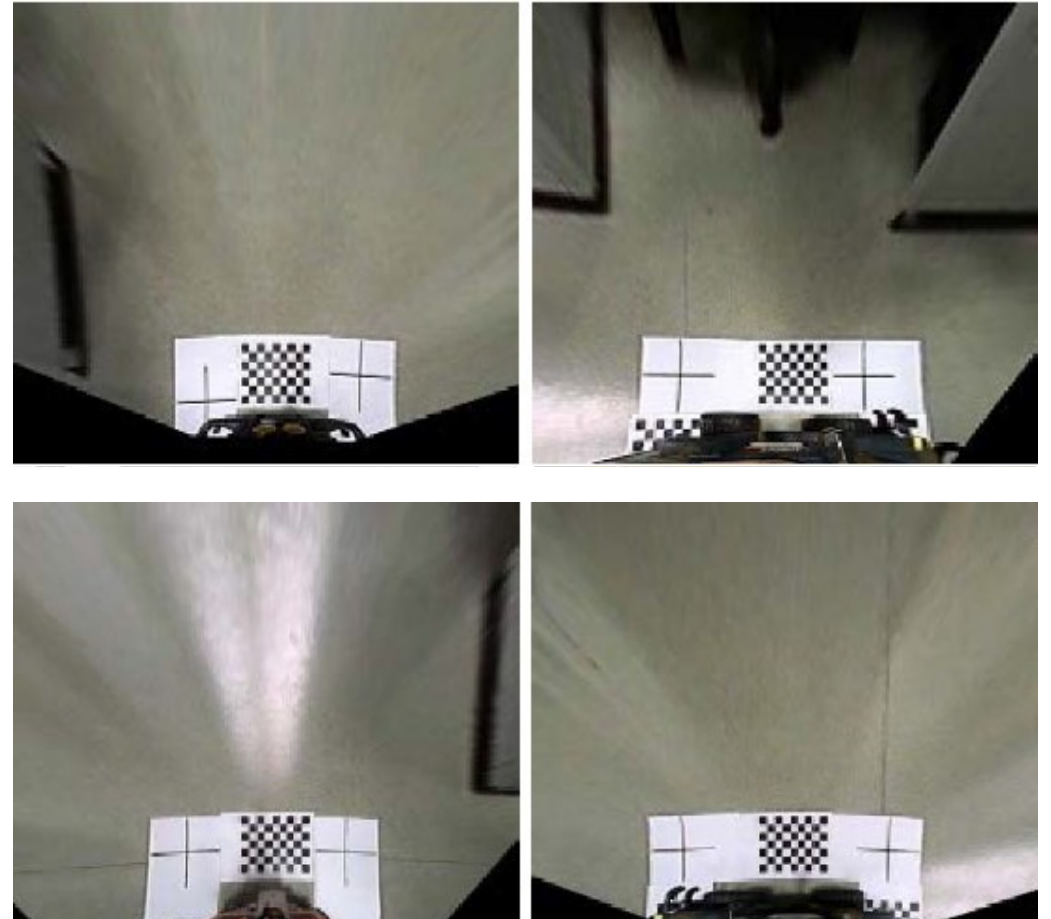
- Kalibracija zasnovana na uzorcima
- Ispravljena distorzija sa slika
- Uklonjeno izobličenje koje unosi fish-eye leća u području interesa



Kako generirati *surround view*?

Geometrijsko poravnanje

- Nakon toga, provodi se inicijalna transformacija perspektive za svaki LDC-ispravljeni okvir



Kako generirati *surround view*?

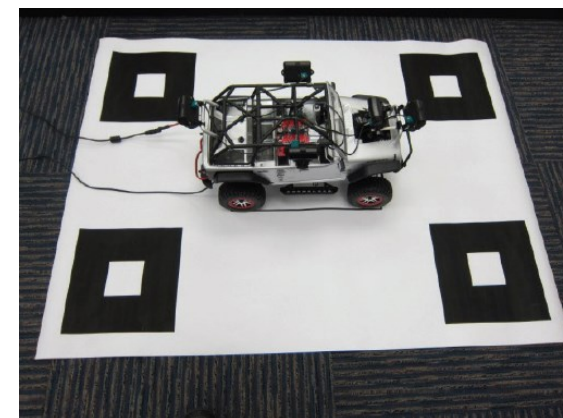
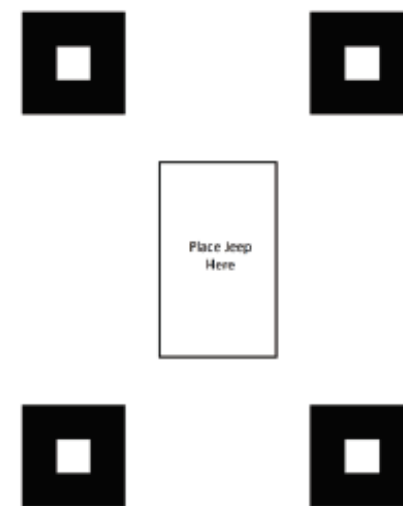
Geometrijsko poravnanje

- Cilj → Pronaći optimalnu matricu transformacije perspektive za svaki okvir
- Konačno → kreira se *look-up-table* (LUT) kako bi se zapisali LDC parametri i informacija o transformaciji perspektive
- Nakon što je dobivena LUT, sprema se u memoriju i koristi se za vrijeme postupka sinteze složenog pogleda, u kojem se kreira konačni 2D *surround view* izlaz

Kako generirati *surround view*?

Geometrijsko poravnanje

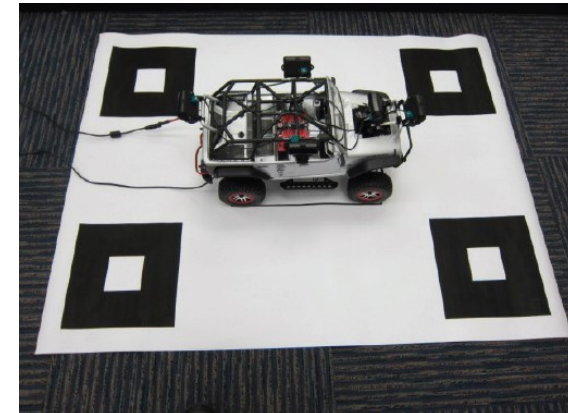
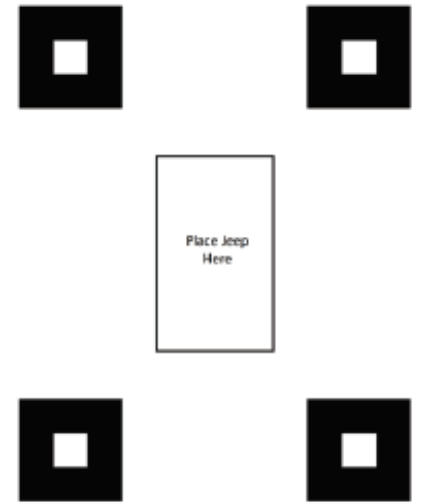
- Za sintezu (spajanje) slika u jednu zajedničku sliku bitno je da u preklapajućim područjima dviju susjednih kamera postoji nekakav objekt na osnovu kojeg će se moći odraditi uparivanje (spajanje, *stitching*) dviju slika
- Ideja → nakon što imamo ključne točke i njihove deskriptore, onda znamo koje točka s jedne slike odgovara određenoj točki (piskelu) s druge slike u preklapajućim područjima
- Na taj način spajamo slike → pikseli koji su dva puta snimljeni, prikazu se u konačnoj slici samo jednom



Kako generirati *surround view*?

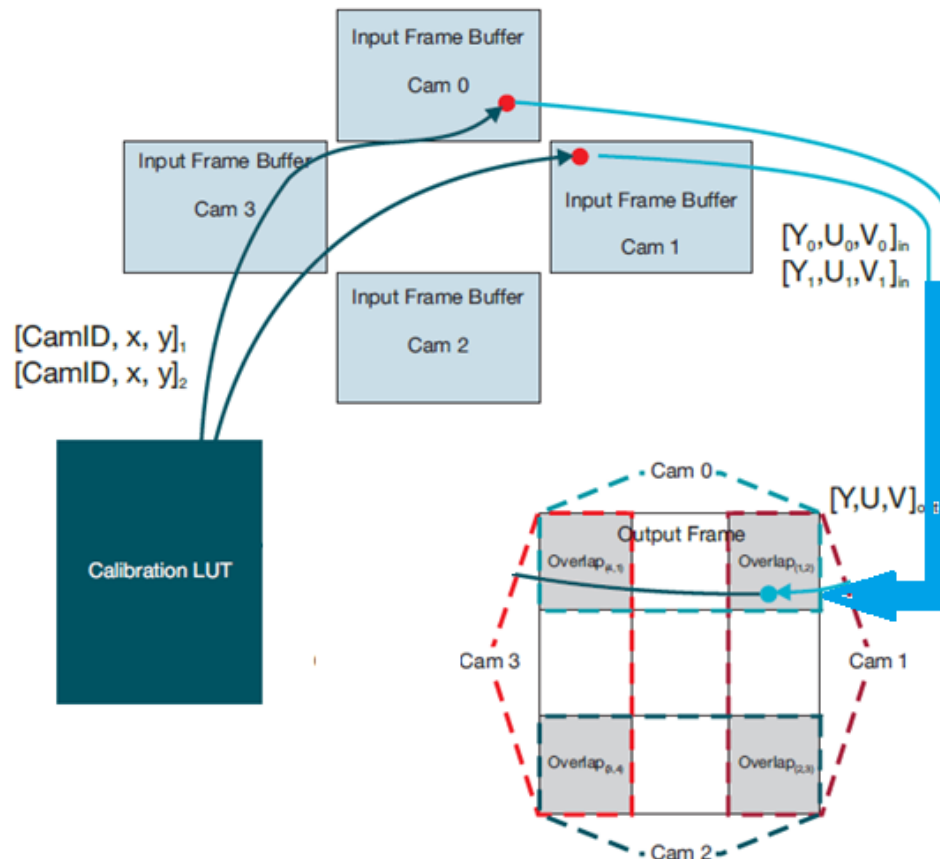
Geometrijsko poravnanje

- Što se konkretno radi? Kako pronaći ključne točke u slikama?
- **Detekcija uglova** kao znalajki kao najjednostavnija metoda
 - *Harris corner detection* se primjenjuje na sliku u preklapajućim područjima susjednih kamera kako bi se pronašla područja interesa
 - Filtriranje „sirovih“ rezultata *Harris corner detection* algoritma kako bi pronašli najjače uglove
 - Računanje BRIEF deskriptora za svaku značajku ugla kako bi pronašli odgovarajuće (*matching*) uglove s dviju susjednih kamera koristeći BRIEF rezultate



Kako generirati *surround view*?

Što nam je poznato nakon geometrijskog poravnanja?



Kako generirati *surround view*?

Fotometrijsko poravnanje

- Zašto postoji razlika u svjetlini/boji istog objekta na slikama s različitih kamera?
 - Različite osvjetljenosti scene
 - Automatska ekspozicija kamere (*auto exposure* – AE)
 - Automatsko balasniranje bijele boje (*auto white balance* – AWB)



Kako generirati *surround view*?

Fotometrijsko poravnanje

- Ako se ne vodi računa o tome, slika složenog pogleda odozgor može sadržavati značajne fotometrijske razlike
- Potrebno je ujednačiti ukupnu svjetlinu i boju različitih pogleda
 - Da konačni pogled izgleda kao da je dobiven jednom kamerom postavljenom iznad vozila



Kako generirati *surround view*?

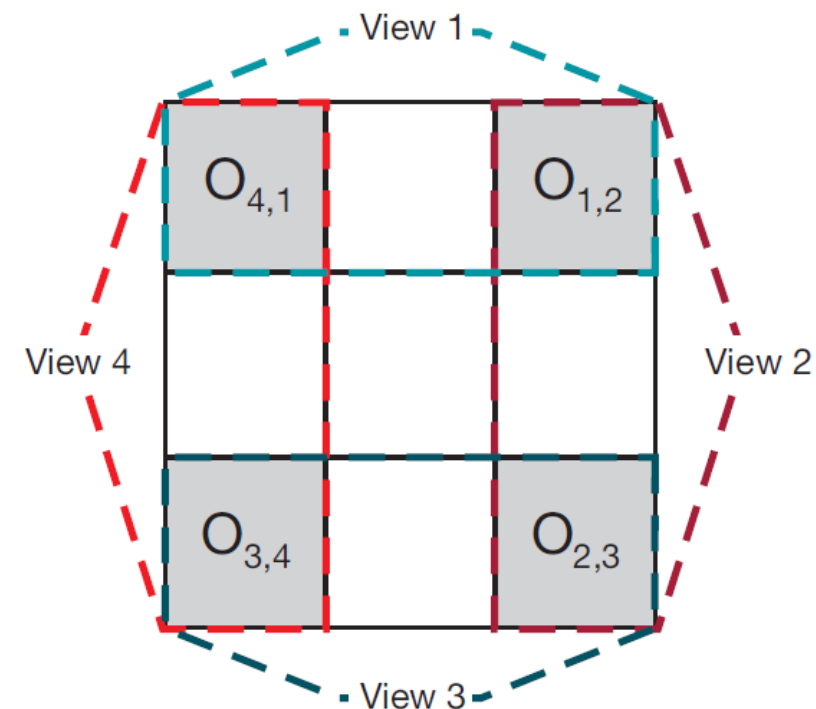
Fotometrijsko poravnanje

- Da bi se ujednačile ukupna svjetlina i boja različitih pogleda
 - Dizajnira se globalna korekcijska funkcija boje i svjetline za svaki pogled, takav da odstupanja u preklapajućim područjima budu minimizirana
- Pretpostavimo da je pravilno geometrijsko poravnanje primijenjeno na ulazne okvire
- Kako izgleda složeni *surround view* pogled ???

Kako generirati *surround view*?

Fotometrijsko poravnanje

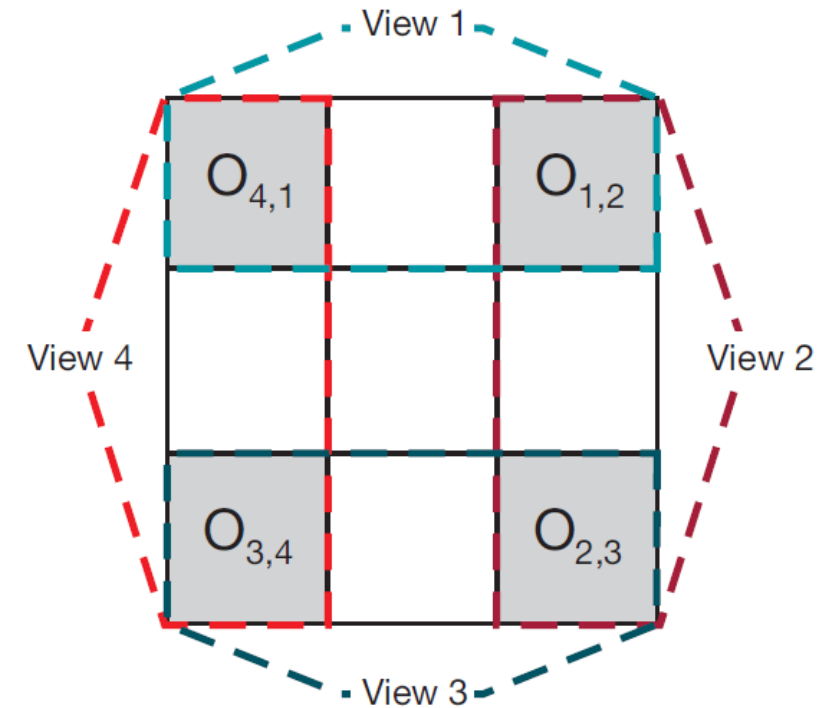
- Pogledi i preklapajuća područja u složenom pogledu odozgor
- Složeni pogled odozgor sastoji se od podataka sa sva 4 ulazna okvira (sa sve 4 kamere)
- $O_{m,n}$ je preklapajuće područje između pogleda dviju susjednih kamera
- Podaci slike u preklapajućim područjima se koriste za proračun funkcije koja će se koristiti za fotometrijske korekcije



Kako generirati *surround view*?

Fotometrijsko poravnanje

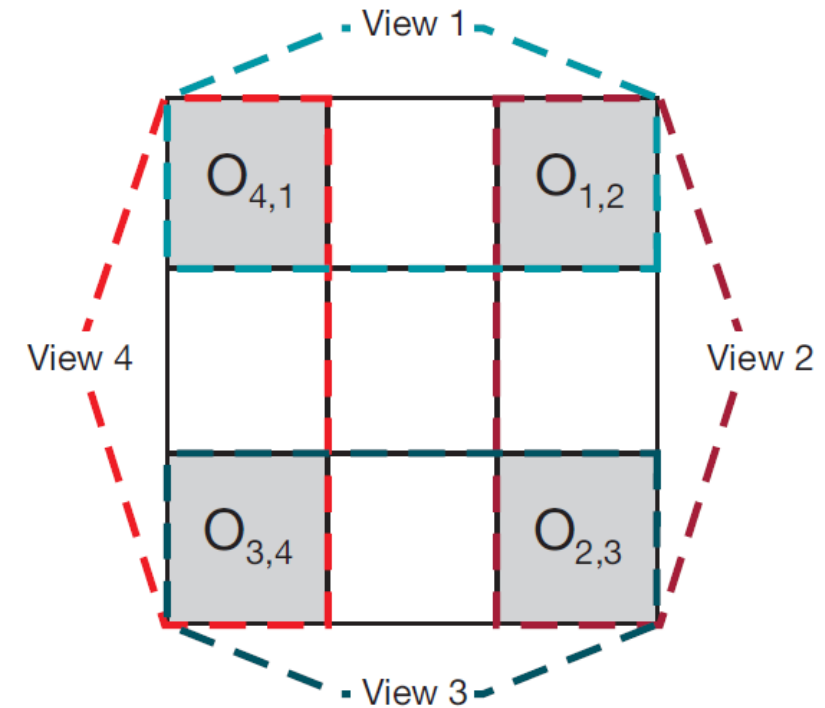
- Na svakoj lokaciji u dijelovima $O_{m,n}$ dostupna su dva piksela
 - Onaj iz pogleda m
 - Njegov prostorni parnjak iz pogleda n
- Za fotometrijsku analizu koriste se ti parovi piksela da se procijeni globalna fotometrijska korekcijska funkcija



Kako generirati *surround view*?

Fotometrijsko poravnanje

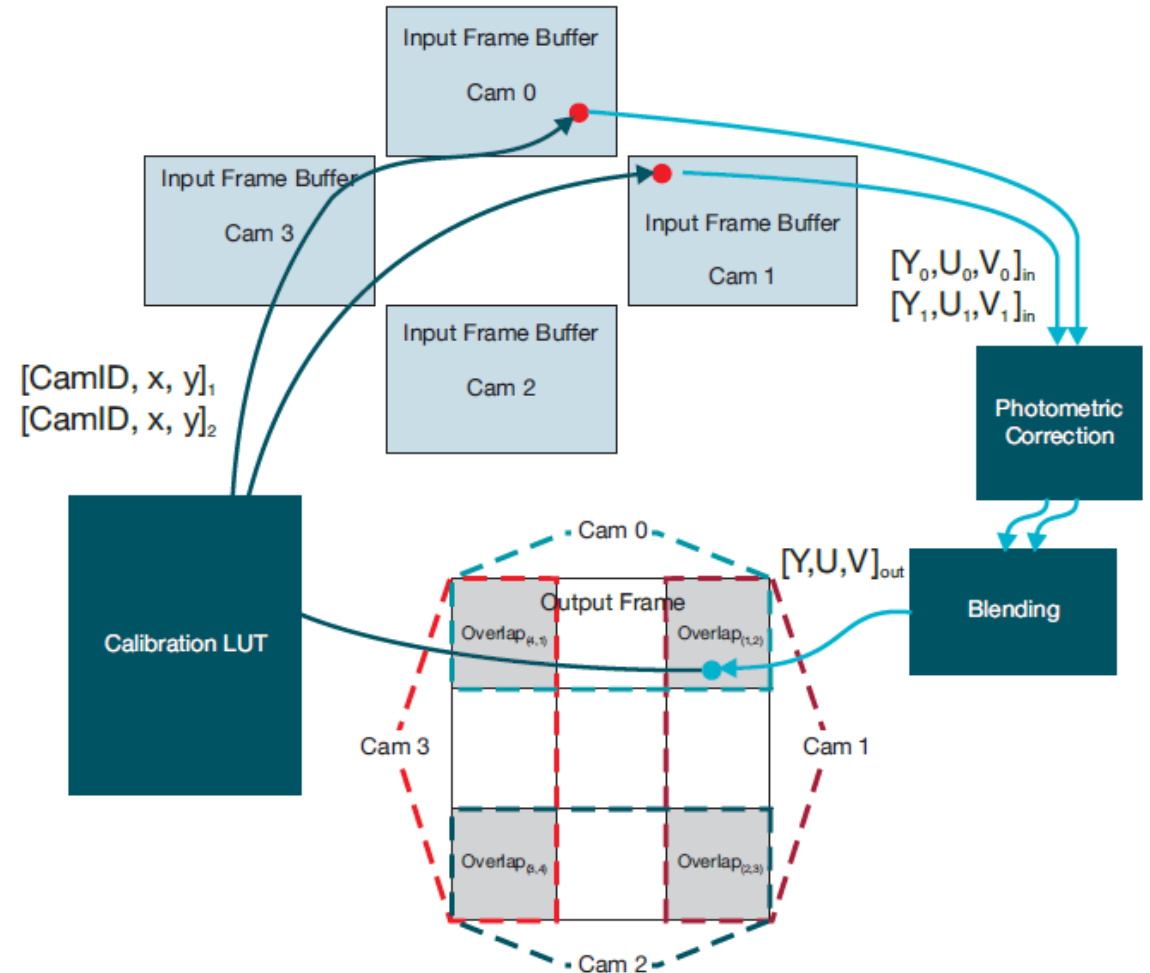
- Npr. za RGB format ulaznih podataka
 - Procjenjuje se *tone mapping* funkcija za svaku pojedinu komponentu R, G, B
 - Minimizira se ukupna srednja kvadratna pogreška odstupanja vrijednosti piksela u svim preklapajućim područjima $O_{m,n}$
 - Odstupanje je definirano kao razlika vrijednosti piksela s kamere m i vrijednosti njegovog parnjaka sa slike n
 - Da bi se ubrzao proračun → *downsampling* preklapajućih područja koristeći *block averaging* prije samog računanja greške



Kako generirati *surround view*?

Sinteza složenog pogleda

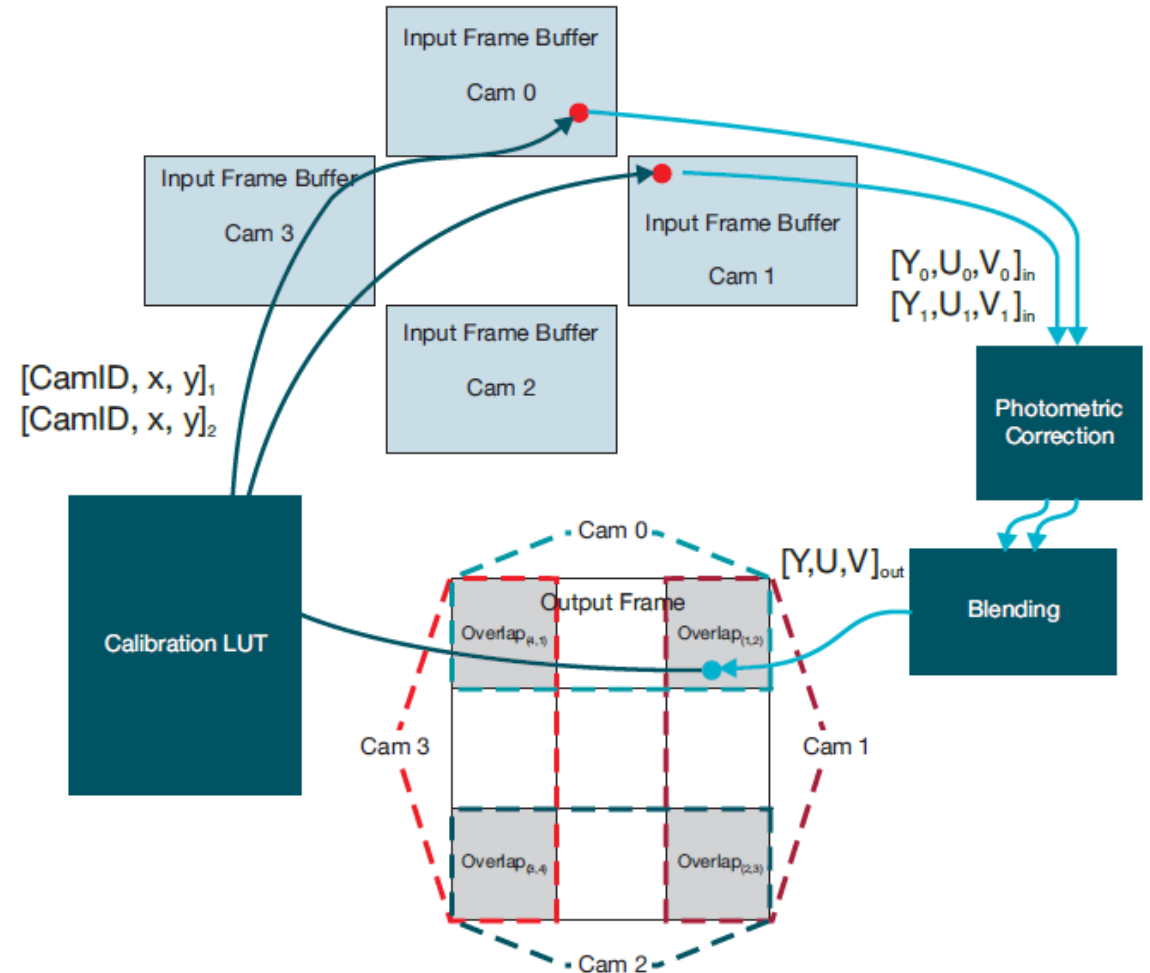
- U preklapajućim područjima izlaznog okvira svaki piksel slike je mapiran s dvije lokacije piksela u dvjema ulaznim slikama
- U tim područjima možemo ili
 - „Pomiješati” (*blending*) podatke iz dviju susjednih slika ili
 - Donijeti binarnu odluku da se koriste podaci iz jedne slike



Kako generirati *surround view*?

Sinteza složenog pogleda

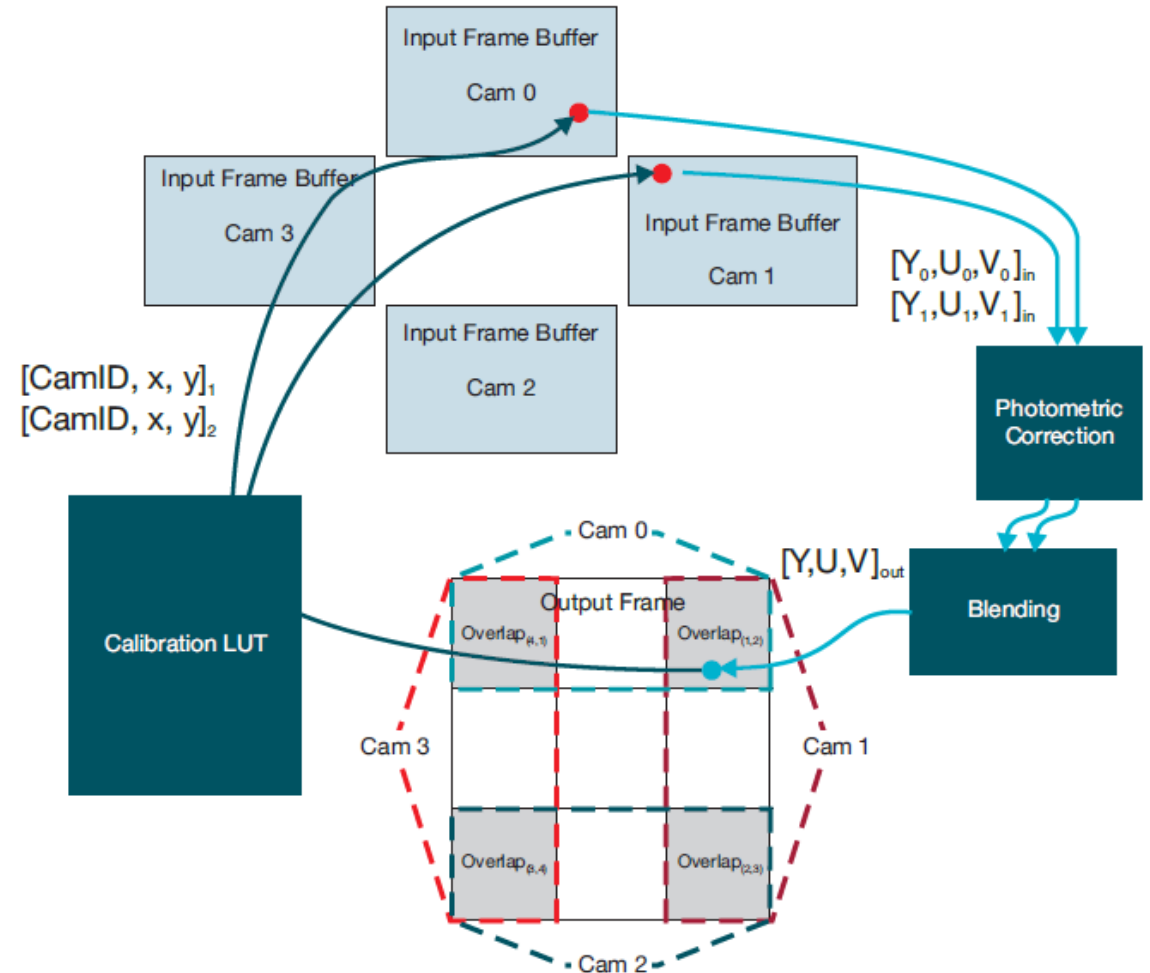
- U ovom primjeru se koristi *blending* podataka (nakon fotometričkog poravnanja)
- Većina nepodudaranja boje i svjetline uklonjeni u postupku fotometrijskog poravnanja
- *Blending* se primjenjuje na fotometrijski ispravljenim podacima da se eliminiraju bilo kakve preostali vidljivi „šavovi“ (podaci za ovo nalaze se u drugoj LUT – *blending* LUT)



Kako generirati *surround view*?

Sinteza složenog pogleda

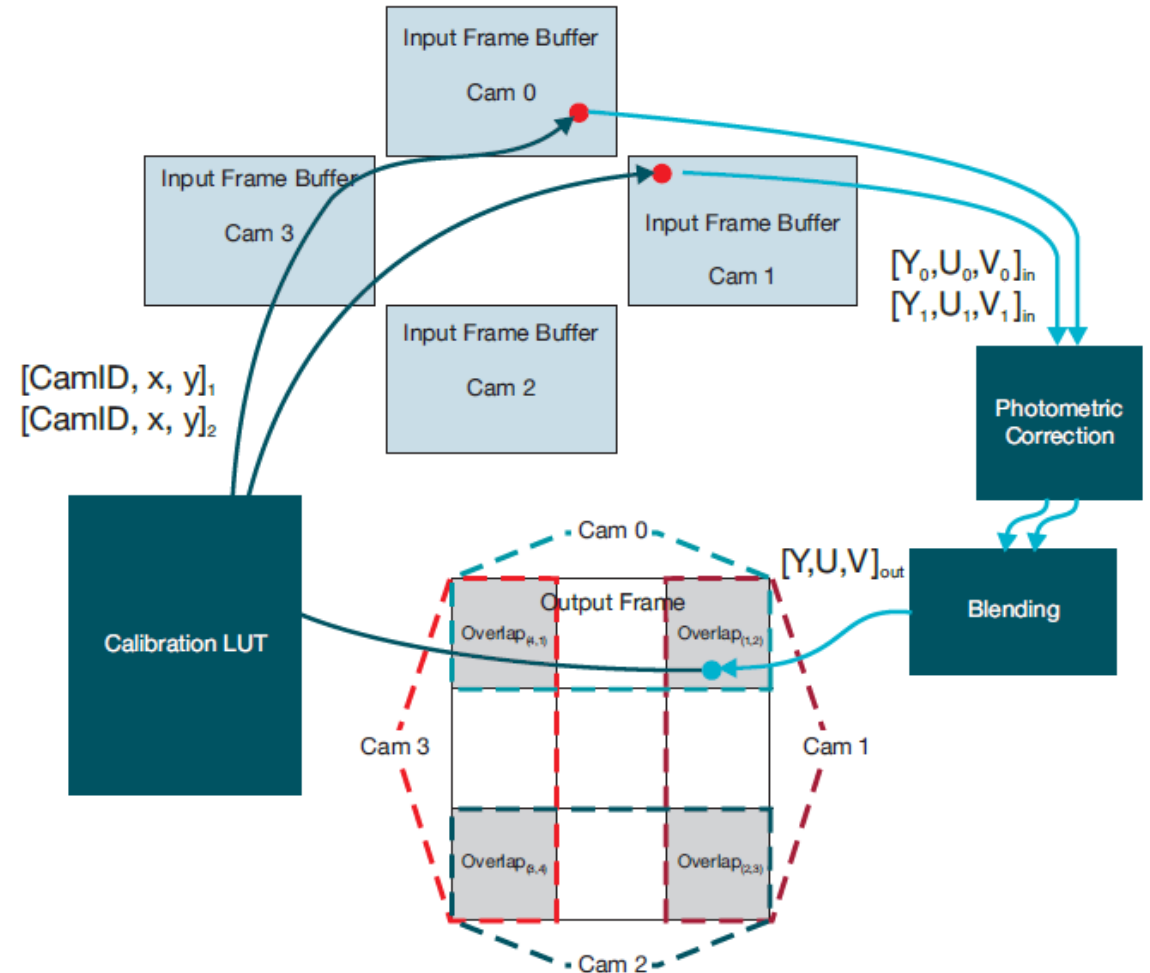
- Izlazni pikseli nakon *blendinga* su generirani kao linearna kombinacija odgovarajućih piksela uz korištenje težinskih faktora iz *blending* LUT tablice
- U nepreklapajućim područjima
 - Samo se jedan ulazni piksel koristi (geometrijska LUT definira koji)
 - Primjenjuje se *tone mapping* iz postupka fotometrijskog poravnanja da se dobije konačni izlazni piksel



Kako generirati *surround view*?

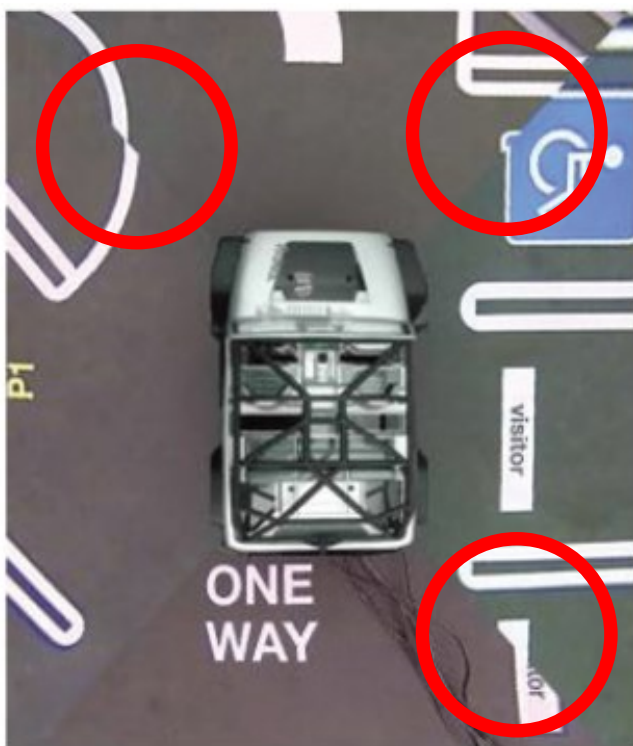
Sinteza složenog pogleda

- Svaki podatak u geometrijskoj LUT specificira
 - ID kamere (tj. indeks ulazne kamere)
 - Koordinate piksela u ulaznom okviru za generiranje izlaznog piksela na trenutnoj lokaciji
- Tek nakon toga se primjenjuju fotometrijska korekcija i *blending* na te piksele, da se generira konačni izlazni piksel



Kako generirati *surround view*?

Kako izgleda konačni izlaz iz algoritma...?



Bez geometrijskog i
fotometrijskog poravnanja



S geometrijskim poravnanjem,
ali bez fotometrijskog



S geometrijskim i
fotometrijskim poravnanjem

Kako generirati *surround view*?

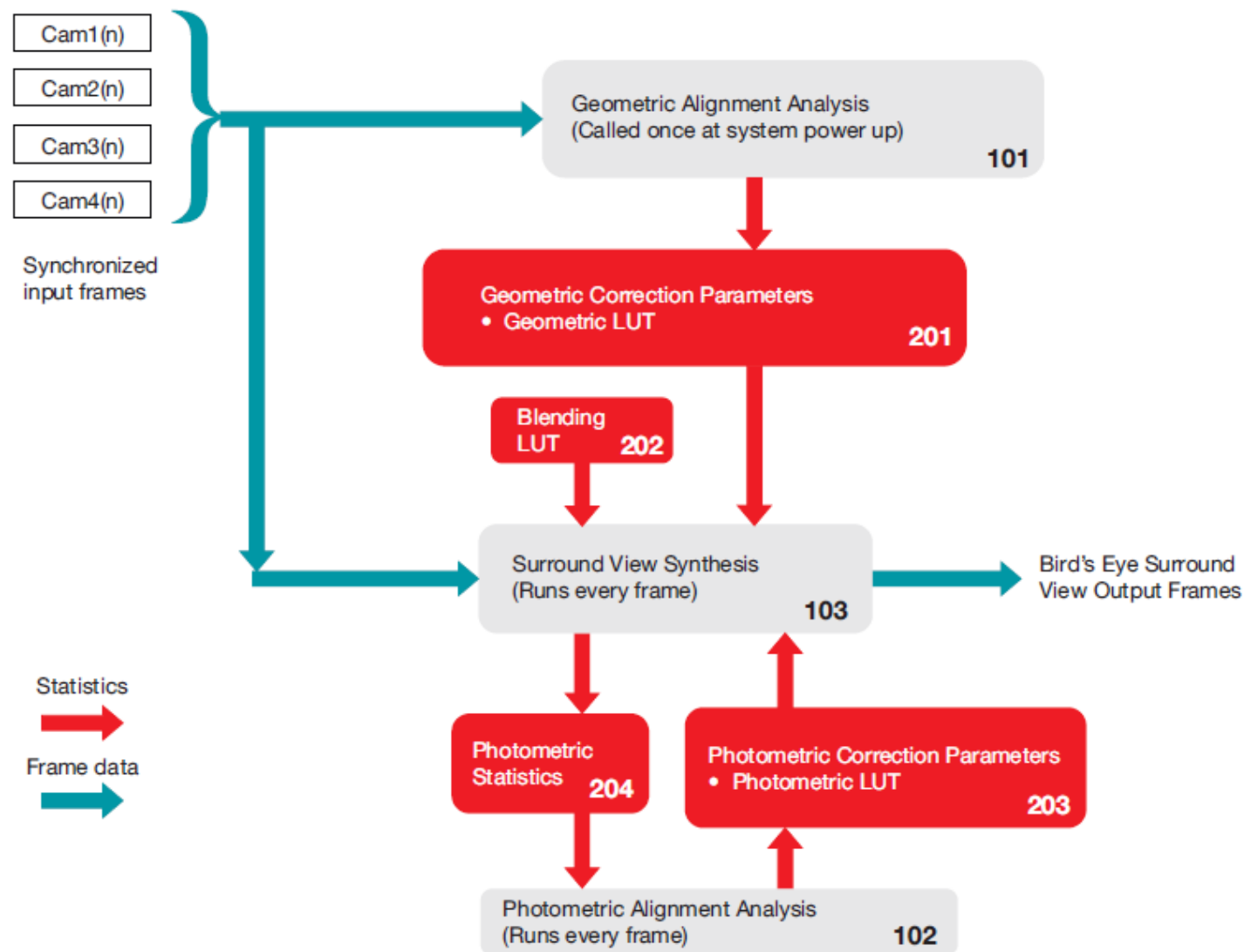
Primjena kod parkiranja vozila



Kako generirati *surround view*?

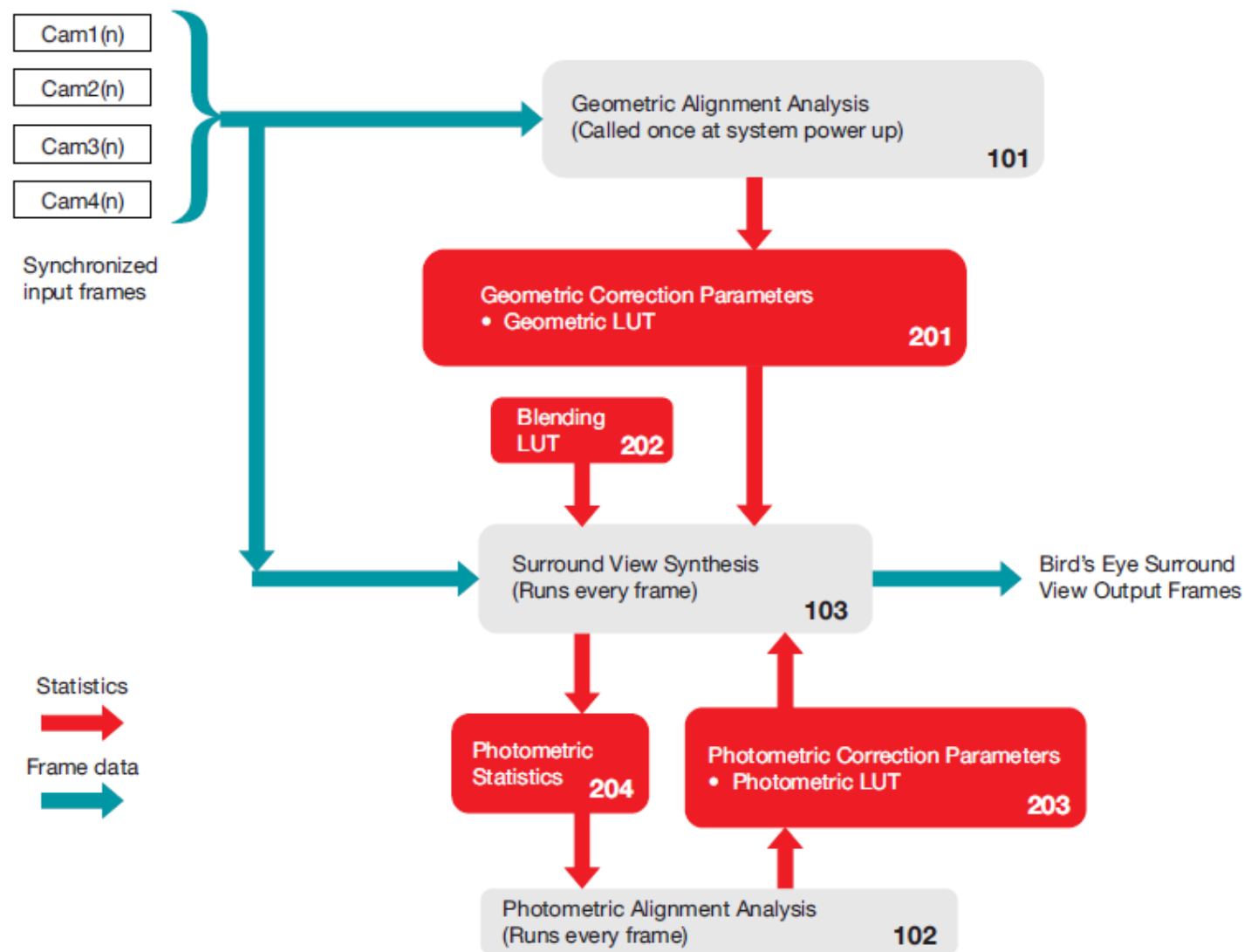
Arhitektura surround view rješenja na Texas Instruments C66x DSP

- Dizajnirana da zadovolji *real-time* performanse i zahtjeve na memoriju u ugradbenim (*embedded*) sustavima
- Dijagram toka rješenja prikazan ja slici



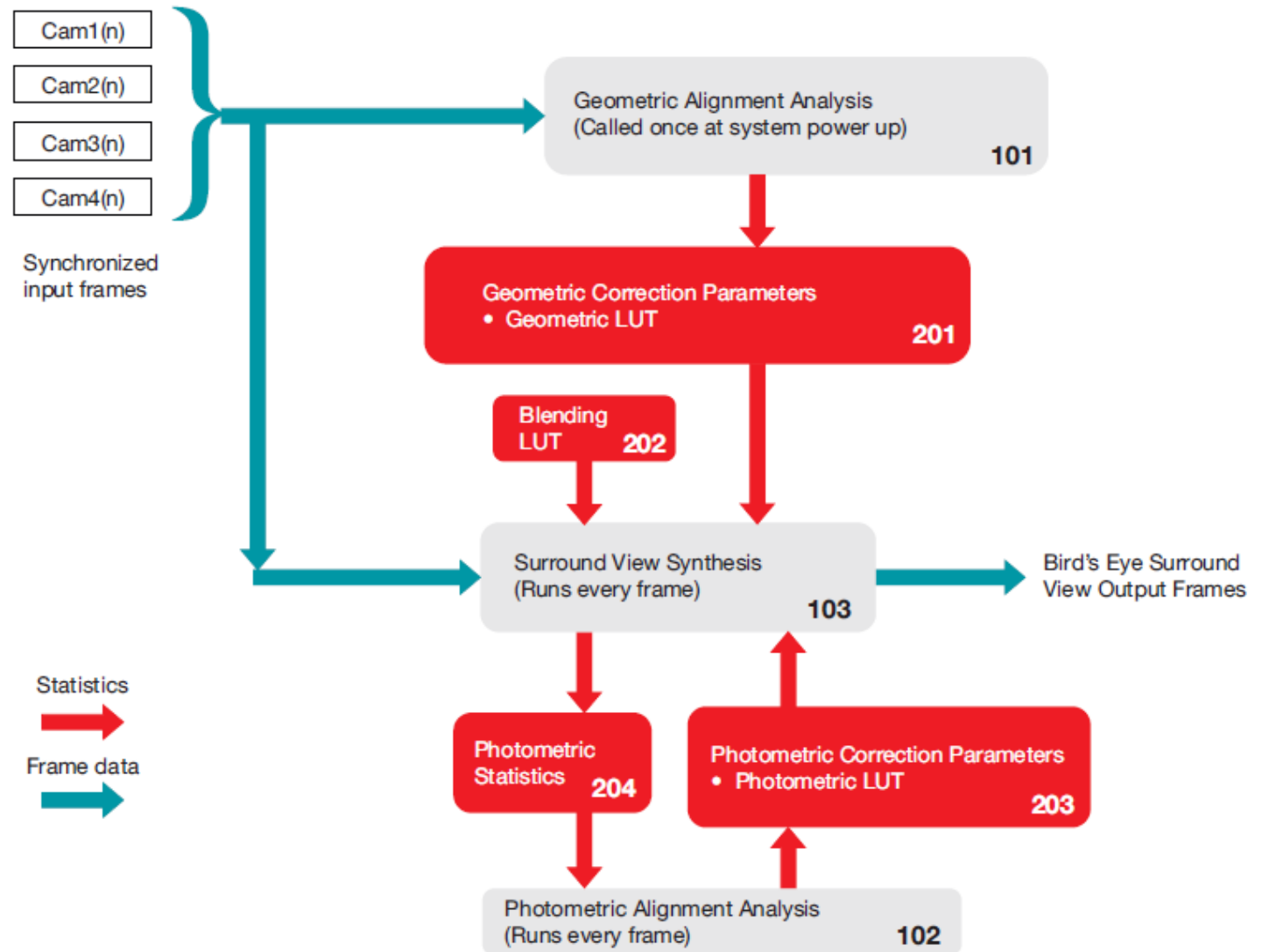
Kako generirati *surround view*?

- O čemu ovisi geometrijska LUT?
 - Samo o lokaciji kamera i ne mijenja se nakon početne instalacije
- Koliko je često potrebno raditi geometrijsko poravnanje?
 - Samo jednom kad je sustav upogonjen



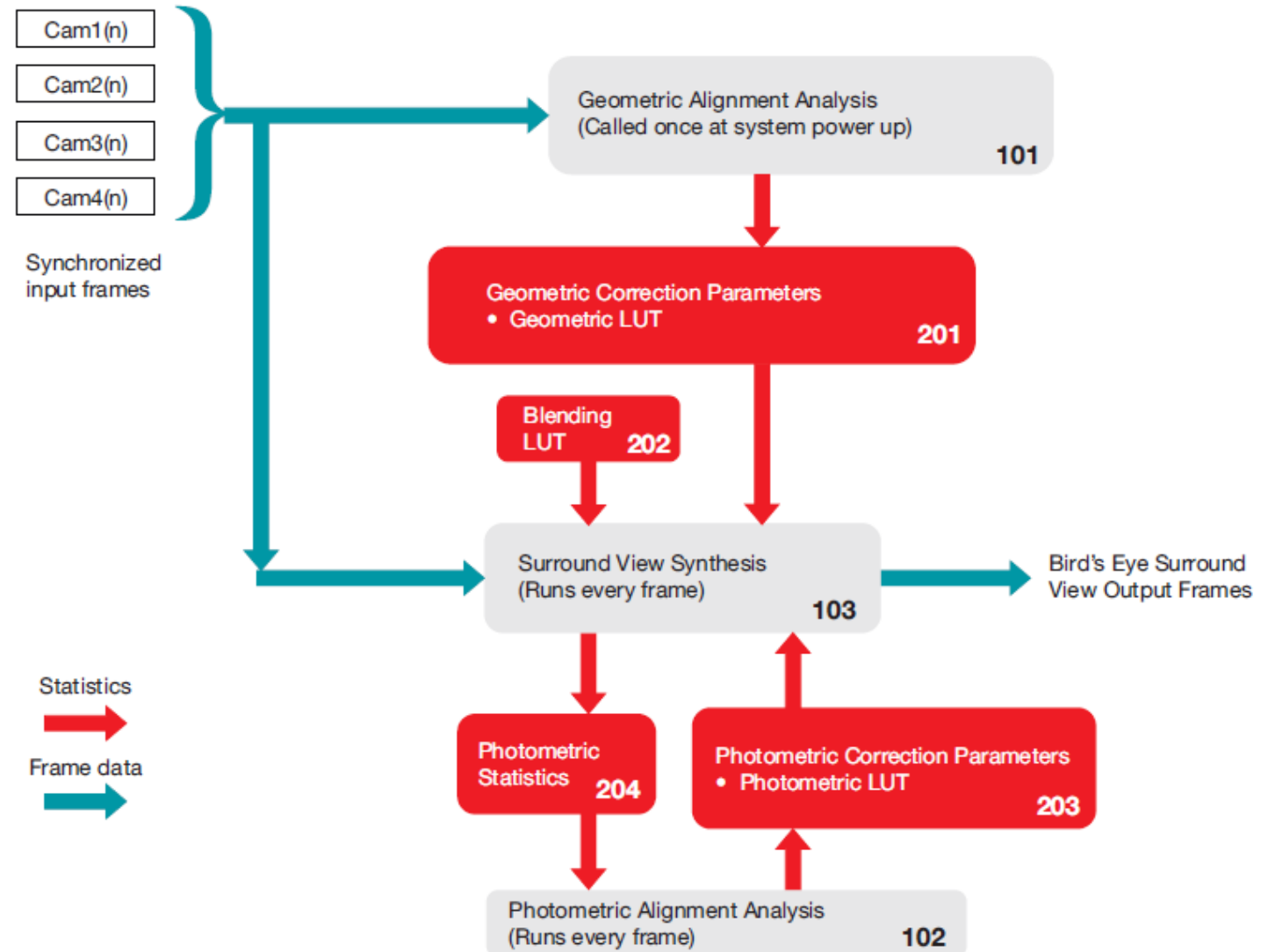
Kako generirati *surround view*?

- SVS funkcija (103) se pokreće za svaki okvir i prima 4 ulaza
- *Fish-eye* okvire sa 4 kamere
- Geometrijsku LUT
- Fotometrijsku LUT
- Blending LUT



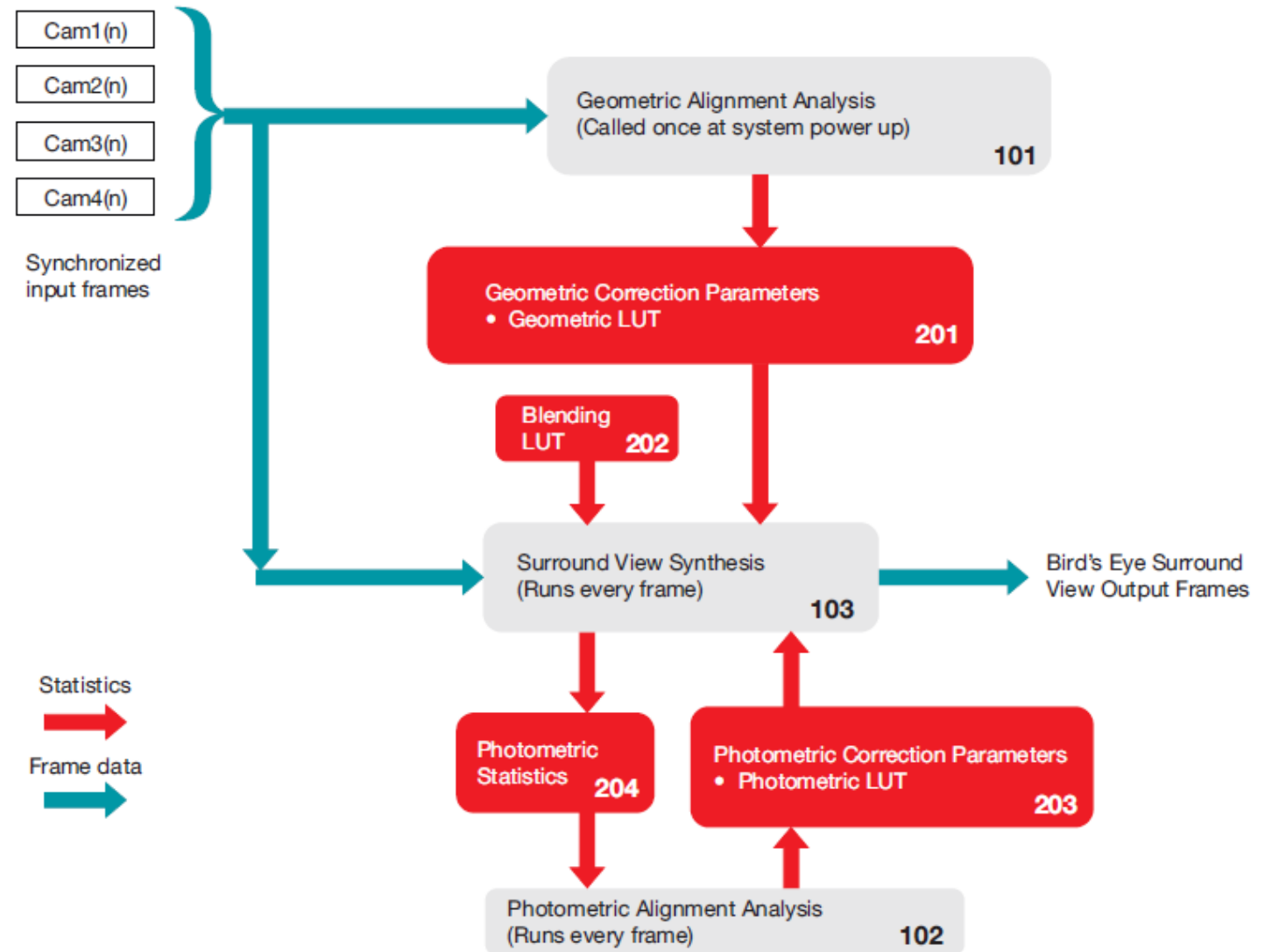
Kako generirati *surround view*?

- SVS funkcija (103) daje 2 izlaza
 - Složeni *surround view* okvir
 - Statistiku za fotometrijsku funkciju za sljedeći trenutak



Kako generirati *surround view*?

- PAA funkcija (102) uzima fotometrijsku statistiku (204) kao ulaz i generira fotometrijske LUT (203) tj. *tone mapping* funkcije za svaku kameru
- Fotometrijska LUT mapira ulaznu vrijednost koja je između 0 i 255 u izlaznu vrijednost istog opsega, kako bi kompenzirala nepodudaranje u boji i svjetlini između četiriju ulaznih okvira

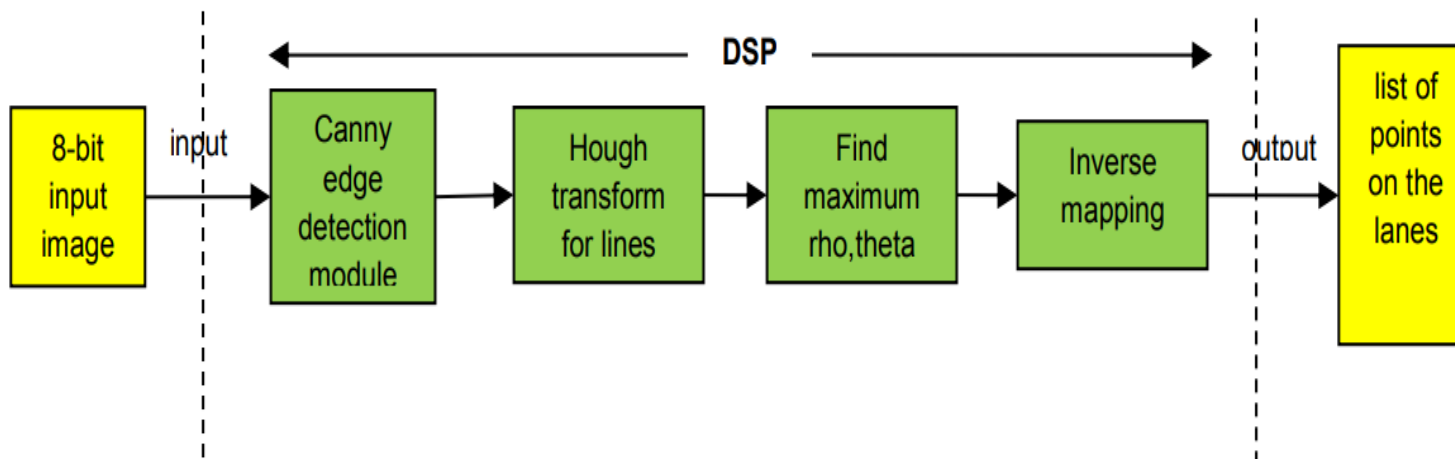


P i t a n j a ?

Detekcija vozne trake i upozorenje o napuštanju vozne trake

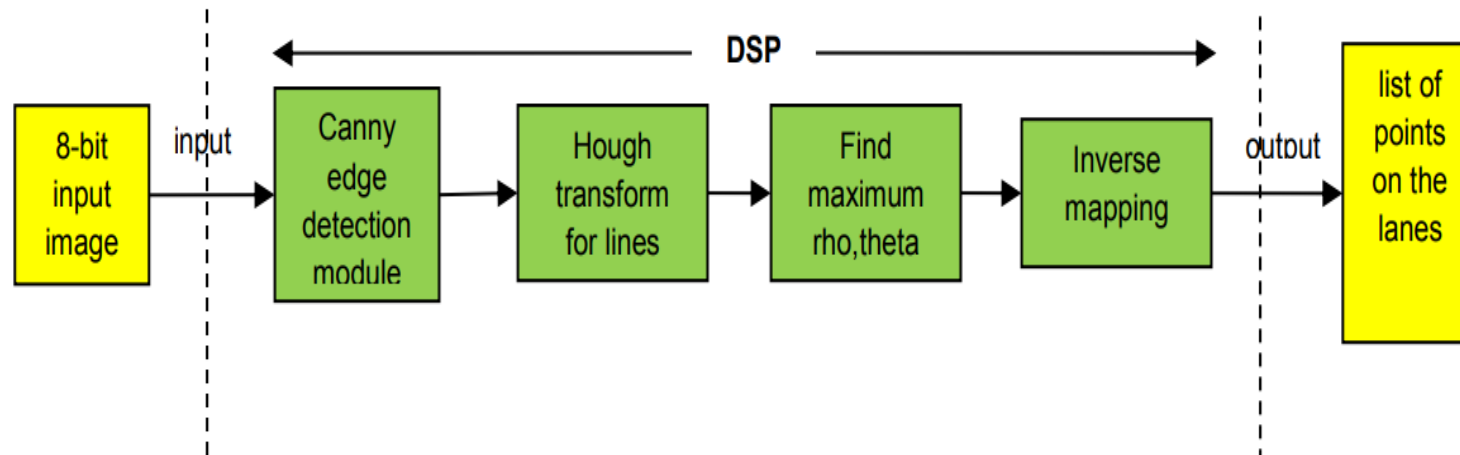
Detekcija vozne trake i upozorenje o napuštanju vozne trake

- Primjer 1 - Detekcija oznaka **vlastite vozne trake** na cesti (za Texas Instruments C66x DSP)
- Upozorenje o napuštanju vozne trake
- Koncept rješenja...



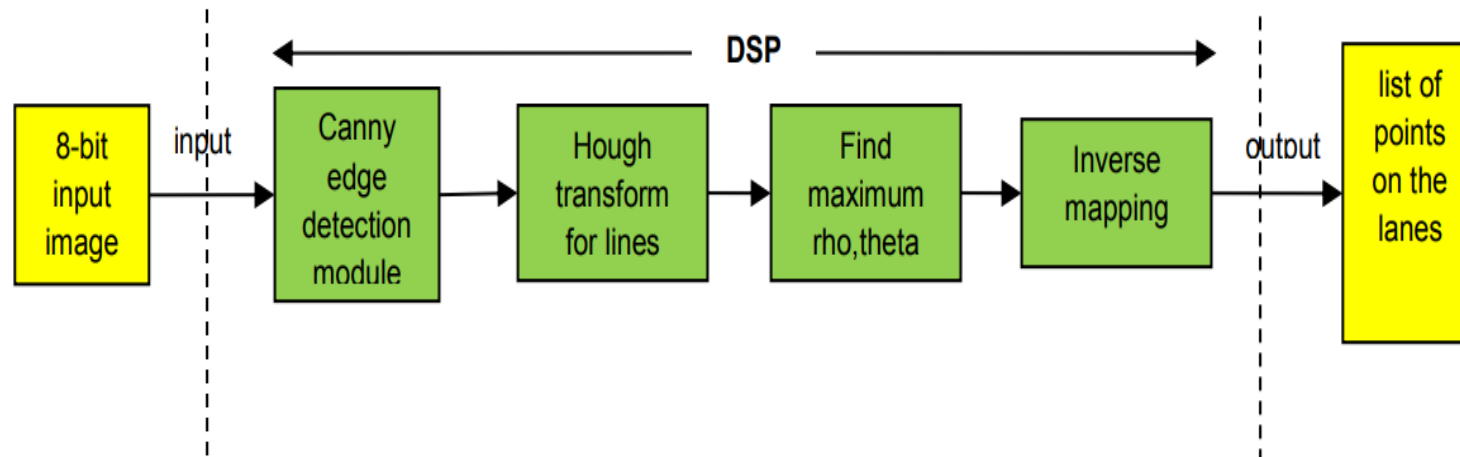
Detekcija vozne trake i upozorenje o napuštanju vozne trake

- Ulaz
 - 8-bitna slika (komponenta svjetline, Y)
 - Koordinate područja interesa (ROI) dijela slike na kojem se nalazi cesta



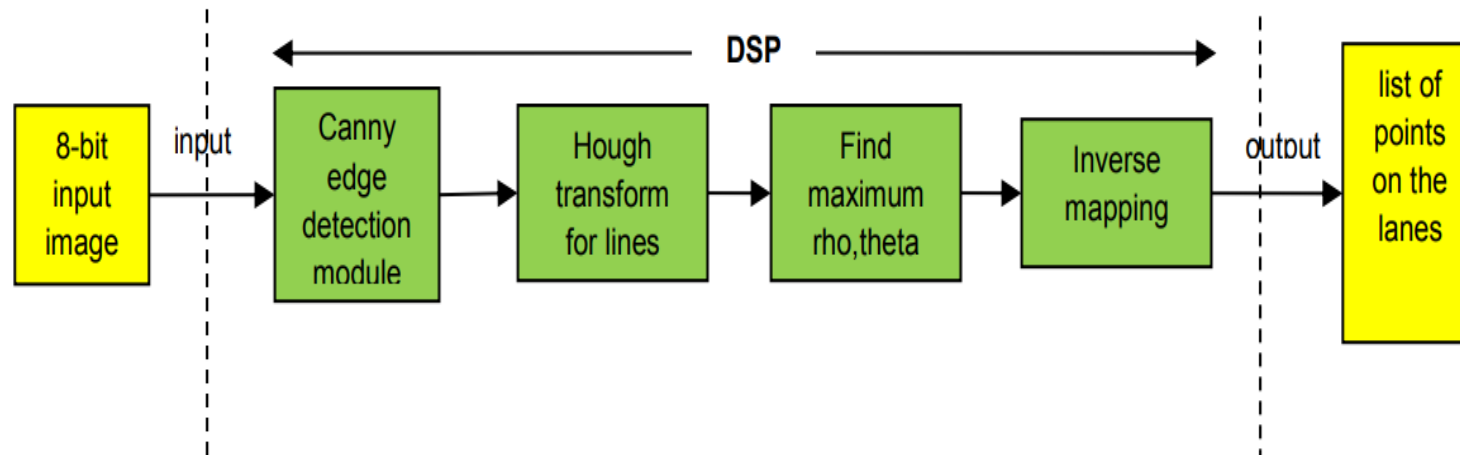
Detekcija vozne trake i upozorenje o napuštanju vozne trake

- Detekcija rubova
 - **Canny metoda za detekciju rubova**
 - Na koji način funkcioniра Canny metoda za detekcija rubova?
 - Zašto bi netko odabrao Canny metodu za detekciju rubova ako je složenija od ostalih?

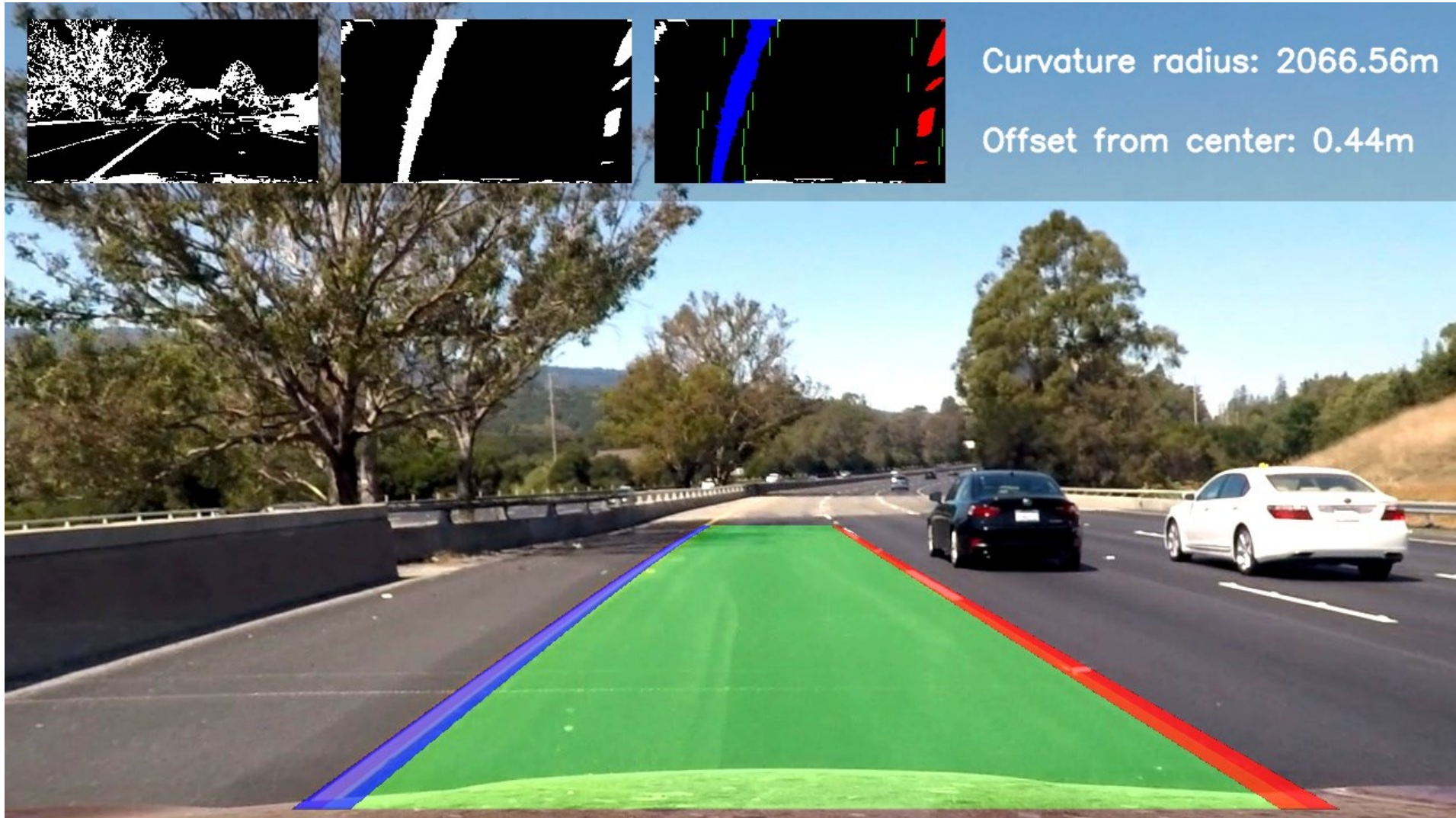


Detekcija vozne trake i upozorenje o napuštanju vozne trake

- Detekcija linija
 - **Hough-ova transformacija** za detekciju linija → kako funkcioniра?
 - Koristi točke koje su identificirane kao rubovi nakon primjene detekcije rubova Cannyevim operatorom i spaja ih u linije



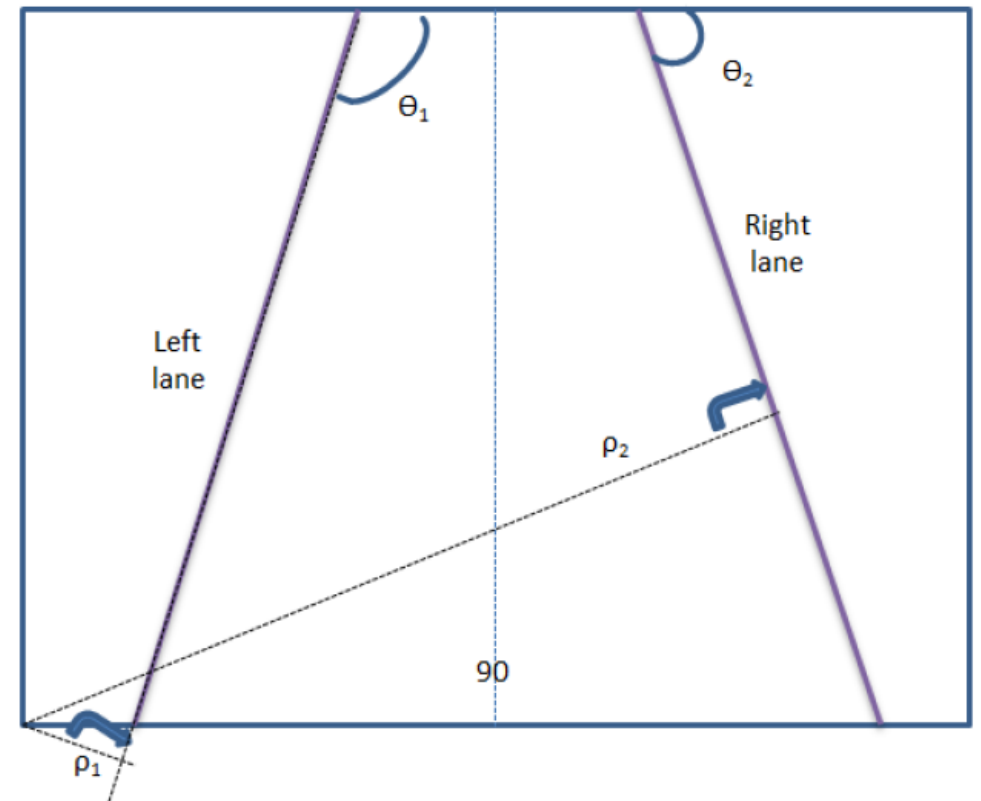
Detekcija vozne trake i upozorenje o napuštanju vozne trake



Detekcija vozne trake i upozorenje o napuštanju vozne trake

Upozorenje o napuštanju vozne trake (*Lane departure warning*)

- Često sastavni dio *Lane detection* algoritma
- Hough-ova transformacija
 - Prebacuje linije iz prostorne domene (x, y) u polarne koordinate (ρ, Θ)
 - Θ je kut između linije i horizontale
 - ρ duljina okomice na liniju povučena iz ishodišta (udaljenost linije od ishodišta)

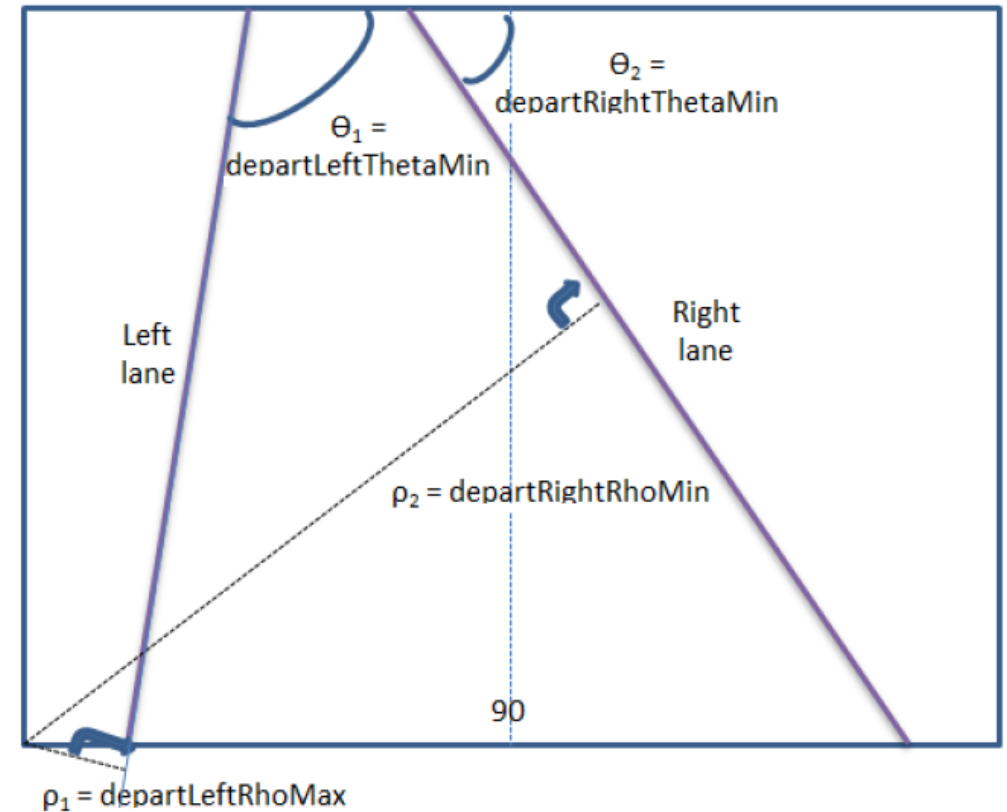


Normalni uvjeti – nema prelaska linije

Detekcija vozne trake i upozorenje o napuštanju vozne trake

Prelazak lijeve linije

- Kut i udaljenost desne linije (ρ_2 , θ_2) se smanjuju
- Kut lijeve linije (θ_1) se smanjuje
- Udaljenost lijeve linije (ρ_1) se povećava
- Parametri koji se koriste kao pragovi kod odluke o upozorenju prelaska lijeve linije vozne trake
 - *departThetaRightMin*
 - *departThetaLeftMin*
 - *departRhoRightMin*
 - *departRhoLeftMax*

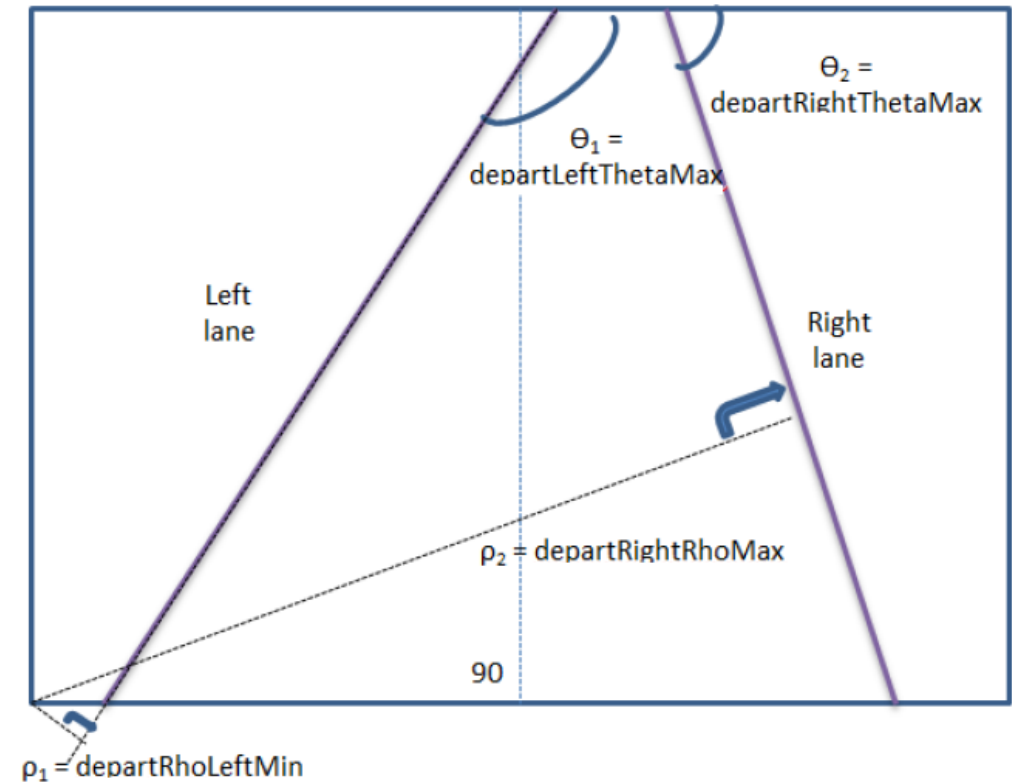


Prelazak lijeve linije - upozorenje

Detekcija vozne trake i upozorenje o napuštanju vozne trake

Prelazak desne linije

- Kut i udaljenost desne linije (ρ_2 , θ_2) se povećavaju
- Kut lijeve linije (θ_1) se povećava
- Udaljenost lijeve linije (ρ_1) se smanjuje
- Parametri koji se koriste kao pragovi kod odluke o upozorenju prelaska desne linije vozne trake
 - *departThetaRightMin*
 - *departThetaLeftMin*
 - *departRhoRightMin*
 - *departRhoLeftMax*



Prelazak desne linije - upozorenje

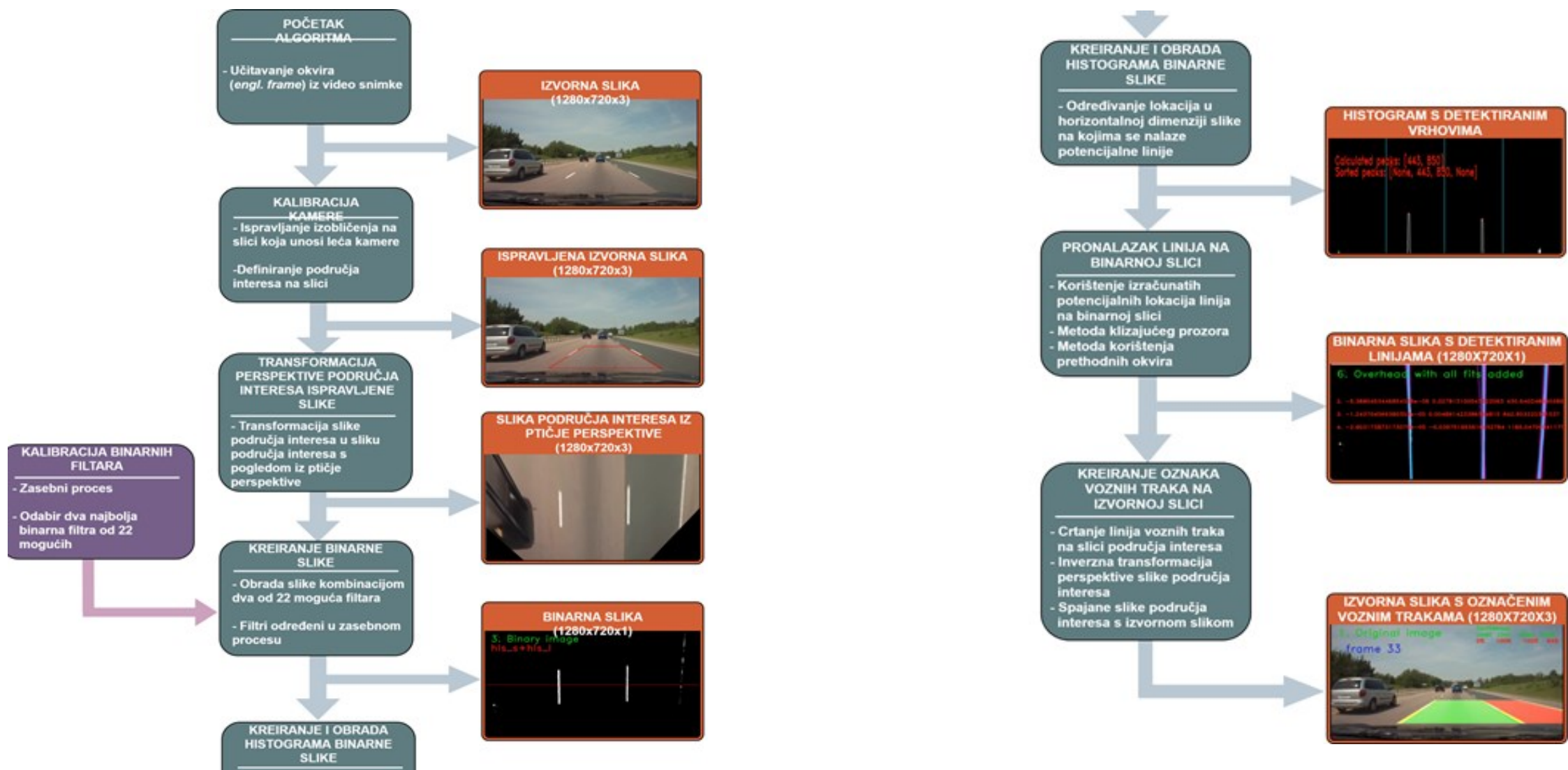
P i t a n j a ?

Detekcija vozne trake i upozorenje o napuštanju vozne trake

- Primjer 2 - Detekcija oznaka vlastite vozne trake i dviju susjednih voznih traka na cesti [7]
- Upozorenje o napuštanju vozne trake



Detekcija vozne trake i upozorenje o napuštanju vozne trake

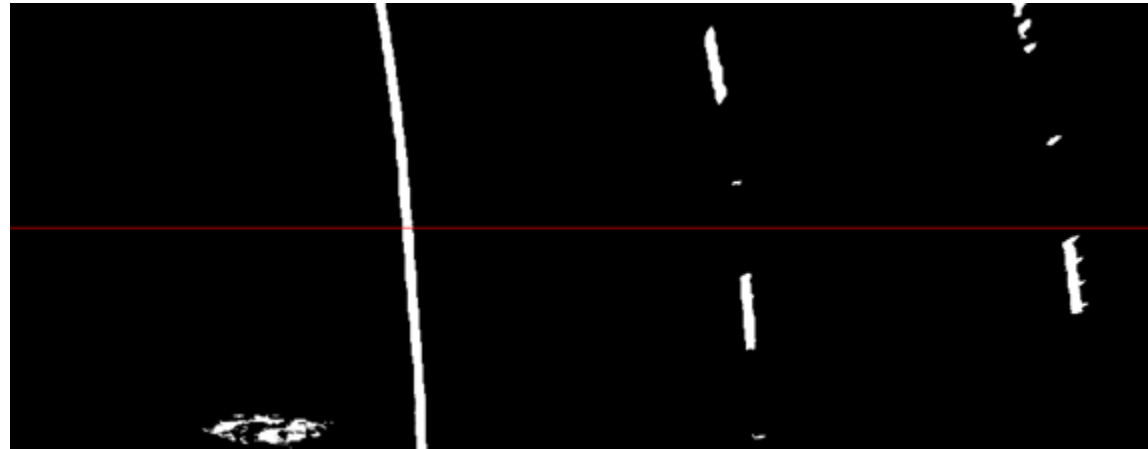


Detekcija vozne trake i upozorenje o napuštanju vozne trake

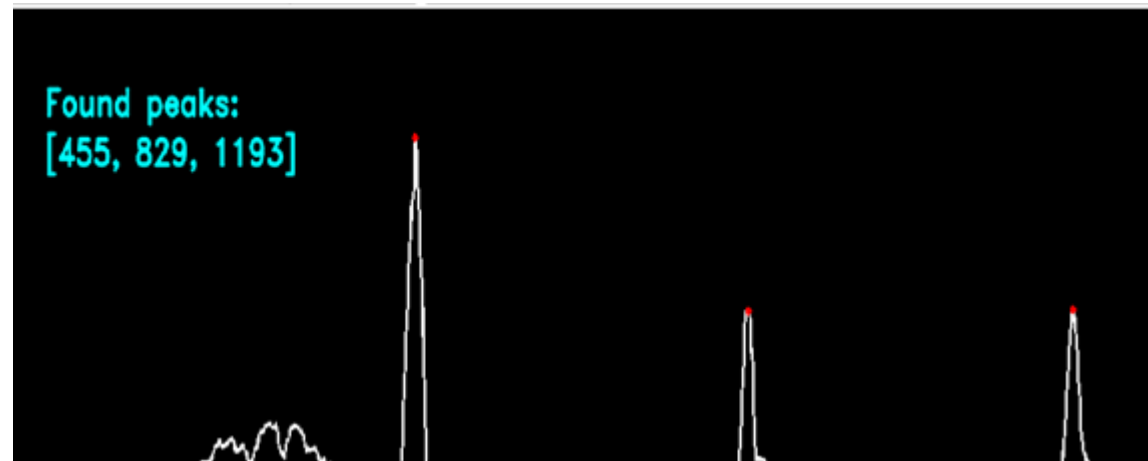
- Primjer 2 - Detekcija oznaka **vlastite vozne trake i dviju susjednih voznih traka** na cesti
- Koraci...
 - Učitavanje video okvira
 - Kalibracija kamere i odabir područja interesa (ROI) u slici
 - Transformacija perspektive područja interesa
 - Kreiranje binane slike
 - Kreiranje i obrada histograma binarne slike
 - Pronalazak linija na binarnoj slici
 - Kreiranje oznaka voznihi traka na izvornoj slici
 - Provjera napuštanja vozne trake

Detekcija vozne trake i upozorenje o napuštanju vozne trake

- Izgled binarne slike

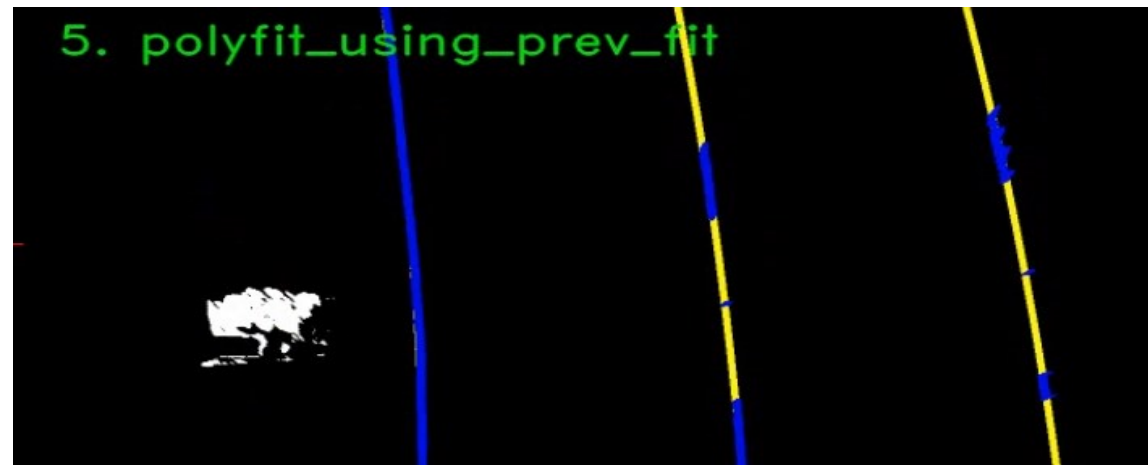
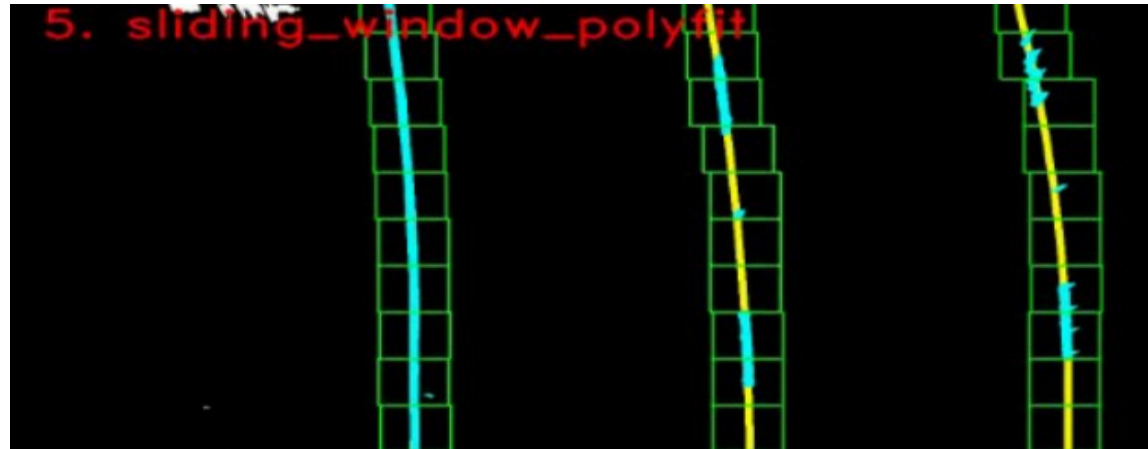


- Histogram binarne slike s detektiranim i označenim vrhovima na slici (crvene oznake)



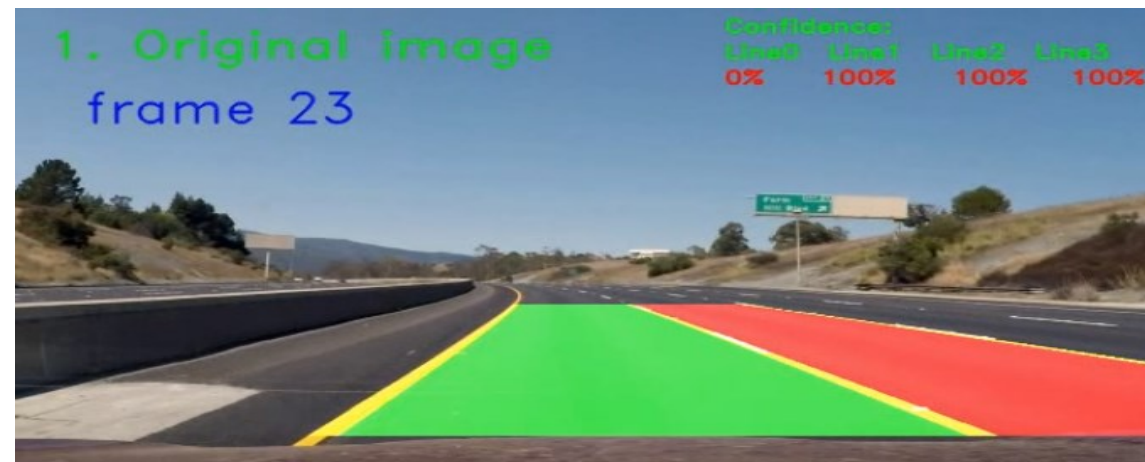
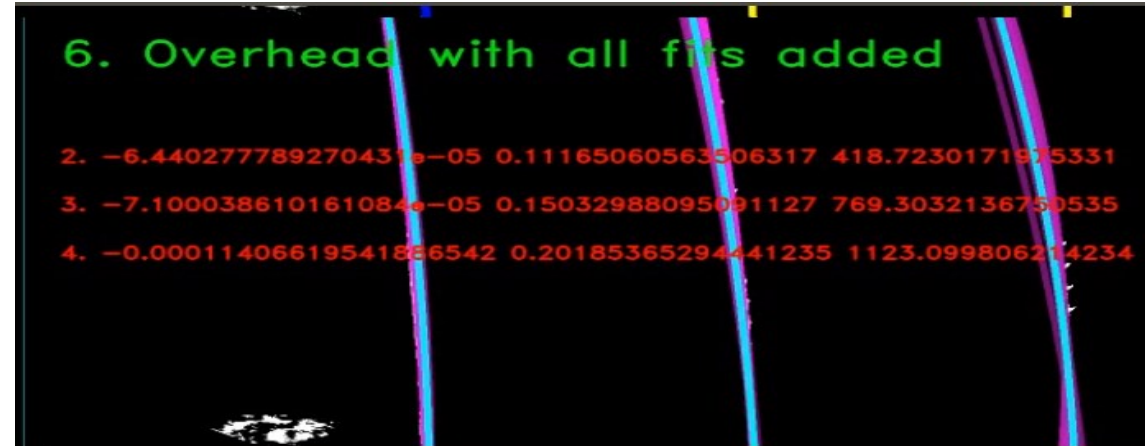
Detekcija vozne trake i upozorenje o napuštanju vozne trake

- Prikaz kreiranja pravokutnika (prozora) na binarnoj slici pomoću metode klizajućeg prozora
- Prikaz rezultata primjene metode detekcije linija s prethodno izračunatim koeficijentima (prošli okvir)



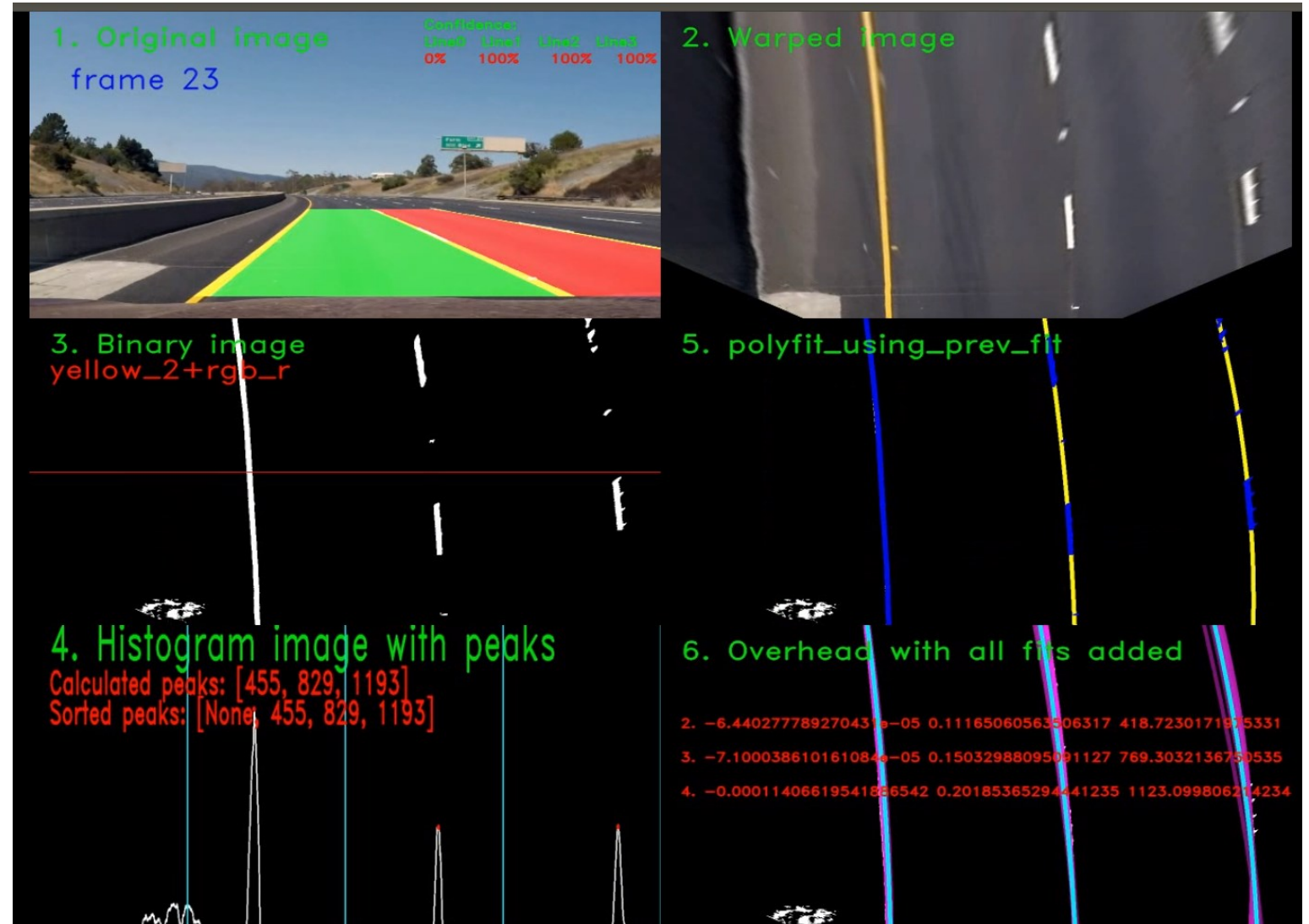
Detekcija vozne trake i upozorenje o napuštanju vozne trake

- Prikaz detektiranih linija na binarnoj slici s pripadnim koeficijentima krivulja polinoma koje označavaju linije vozne trake
- Prikaz detektiranih linija i obojanih voznih traka na izvornoj slici



Detekcija vozne trake i upozorenje o napuštanju vozne trake

- Izgled dijagnostičkog prikaza rada kompletnog algoritma za detekciju linija voznih traka na jednom okviru snimke



Detekcija vozne trake i upozorenje o napuštanju vozne trake

- Primjer 2 - Detekcija oznaka vlastite vozne trake i dviju susjednih vozni traka na cesti
- Kako je implementirana detekcija napuštanje vozne trake i alarmiranje na taj događaj?
 - Nije korištena Hough-ova transformacija → ne znamo parametre linija
 - Iz poznate lokacije kamera u vozilu, poznate prosječne širine vozne trake i poznatih parametara kamere moguće je odrediti gdje moraju biti lokacije linija u histogramu, kako bi vozilo bilo u svojoj traci
 - Čim te linije nisu u nekom definiranom opsegu u histogramu u kojem moraju biti, onda se oglašava alarm

Detekcija vozne trake i upozorenje o napuštanju vozne trake

- Primjer 2 - Detekcija oznaka vlastite vozne trake i dviju susjednih vozni
traka na cesti



Detekcija vozne trake i upozorenje o napuštanju vozne trake



P i t a n j a ?

**Detekcija objekata (pješačka,
prometnih znakova, vozila)**

Detekcija objekata (pješača, prometnih znakova, vozila)

- Detektirati objekte ispred vozila na osnovu slike dobivene pomoću kamere na prednjoj strani vozila
- Primjer rješenja koji slijedi → **treba detektirati različite objekte koji se susreću u prometu**
 - Vozila
 - Pješače
 - Prometne znakove
 - Semafore

i za svaki okvir video zapisa treba pohraniti lokaciju i dimenzije detektiranih objekata
- Rješenje [3] → OpenCV 3.1.0 && dlib biblioteka && Python 3

Detekcija objekata (pješaka, prometnih znakova, vozila)

- **Viola-Jones algoritam za detekciju objekata**
- Izrađeni detektori za pojedine tipove objekata (ukupno 10 njih)
 - Automobil prednja i stražnja strana,
 - Automobil bočna strana,
 - Autobus i kamion prednja i stražnja strana
 - Pješak
 - Prometni znak opasnosti,
 - Prometni znak izričitih naredbi koje karakterizira plava pozadina,
 - Prometni znak izričitih naredbi koje karakterizira crveni rub,
 - Prometni znak obveznog zaustavljanja,
 - Prometni znak raskrižja s cestom s prednošću prolaska
 - Semafor

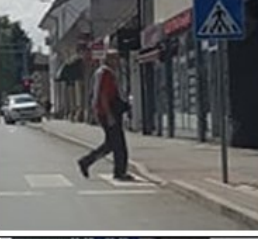
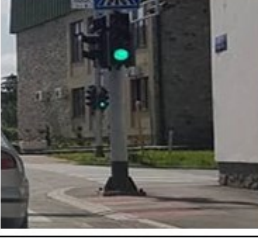
Detekcija objekata (pješača, prometnih znakova, vozila)

- Izrada detektora → Viola-Jones algoritam
 - Pozitivne i negativne slike (vlastite i postojeće) prilikom učenja

Naziv detektora	Primjeri slika korištenih prilikom procesa učenja		
Detektor prednje i stražnje strane automobila			
Detektor bočne strane automobila			

Detekcija objekata (pješaka, prometnih znakova, vozila)

Naziv detektora	Primjeri slika korištenih prilikom procesa učenja		
Detektor prednje i stražnje strane autobusa i kamiona			
Detektor prometnih znakova opasnosti			
Detektor prometnih znakova izričitih naredbi koje karakterizira plava pozadina			
Detektor prometnih znakova izričitih naredbi koje karakterizira crveni rub			

Naziv detektora	Primjeri slika korištenih prilikom procesa učenja		
Detektor prometnog znaka obveznog zaustavljanja			
Detektor prometnog znaka raskrižja s cestom koja ima prednost prolaska			
Detektor pješaka			
Detektor semafora			

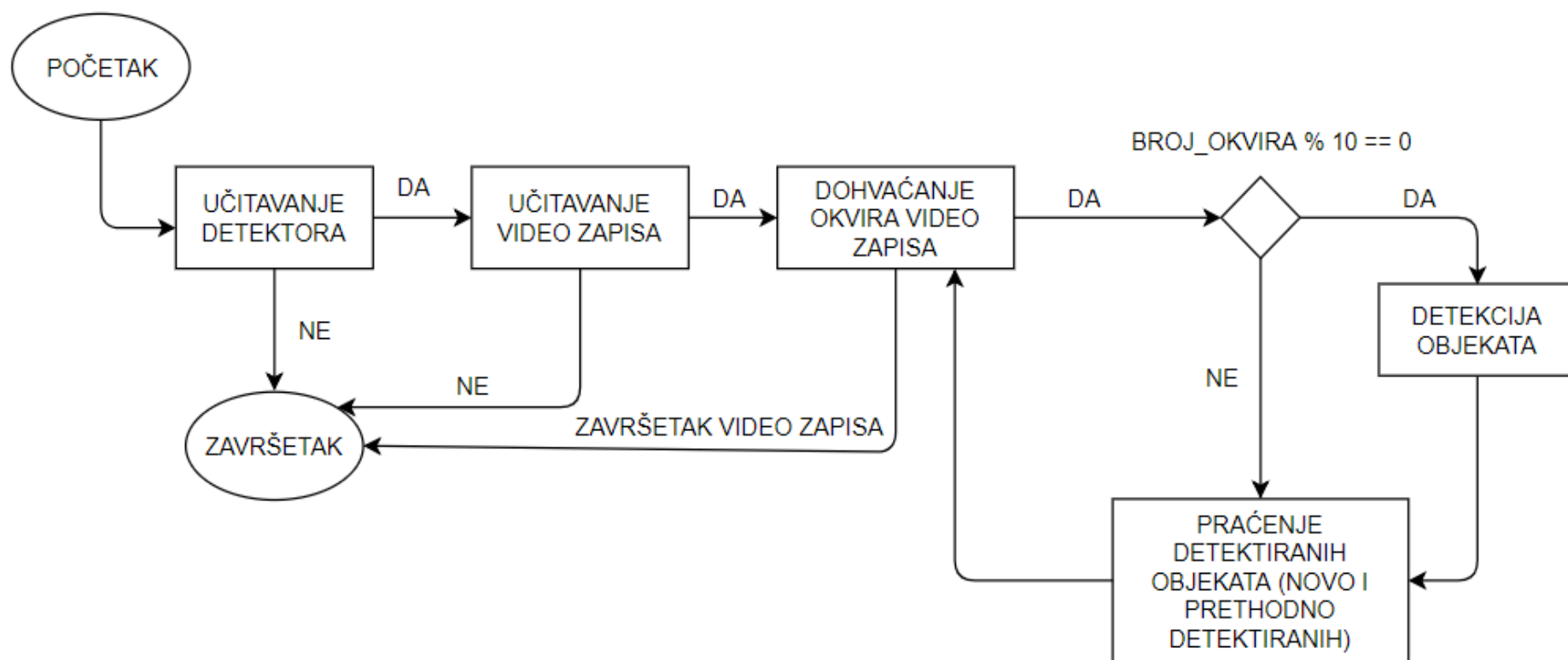
Detekcija objekata (pješaka, prometnih znakova, vozila)

- Informacije o procesu učenja za svaki od ulaznih detektora

Detektor	Broj pozitivnih slika	Broj negativnih slika	Broj kaskada	Dimenzija izlaznog uzorka [piksela]	Vrijeme trajanja učenja [h]
Prednje i stražnje strane automobila	1700	4300	24	24 x 24	24
Bočne strane automobila	1200	4300	24	30 x 15	24
Prednje i stražnje strane autobusa i kamiona	600	4300	24	24 x 32	22
Prometnih znakova opasnosti	2000	3000	15	24 x 24	16
Prometnih znakova izričitih naredbi koje karakterizira plava pozadina	1000	4300	17	24 x 24	16
Prometnih znakova izričitih naredbi koje karakterizira crveni rub	2000	3000	19	24 x 24	20
Prometnog znaka obveznog zaustavljanja	580	4300	10	24 x 24	12
Prometnog znaka raskrižja s cestom koja ima prednost prolaska	1500	3000	19	24 x 24	20
Pješaka	1488	3761	24	12 x 36	24
Semafora	1000	4300	26	5 x 15	36

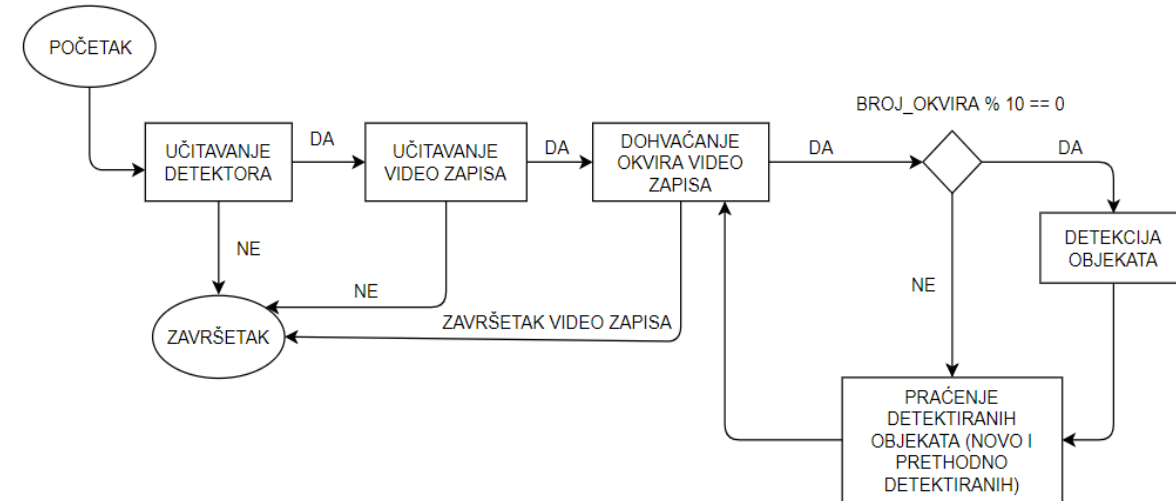
Detekcija objekata (pješača, prometnih znakova, vozila)

- Izrada računalnog algoritma za detekciju i praćenje objekata u videu



Detekcija objekata (pješača, prometnih znakova, vozila)

- Detektira objekte na svakom 10. okviru videa
- Na ostalim okvirima prati detektirane objekte
- Zašto baš tako?
 - Pretpostavka je bila da je potrebno obraditi barem 10 slijednih okvira video zapisa kako bi se pojavio novi objekt u video zapisu
- Detekcija objekata → detektori i OpenCV
- Praćenje objekata → OpenCV



Detekcija objekata (pješaka, prometnih znakova, vozila)

- Koji algoritmi za praćenje objekata postoje implementirani u OpenCV ?
 - *Meanshift*
 - Nije pogodan – Zašto ?
 - Dimenzija graničnog okvira uvijek identična njegovoj početnoj dimenziji bez obzira na promjenu lokacije objekta (približavanje ili udaljavanje)
 - Ako je objekt izvan slike odnosno okvira video zapisa, njegovo praćenje neće prestati
 - Ako se promijeni osvjetljenje između slijednih okvira video zapisa, tada najčešće dolazi do pogrešnog praćenja traženih objekata
 - *Camshift*
 - Promijenjena dimenzija objekta ovisno o njegovu približavanju ili udaljavanju
 - Ali nije pogodan – Zašto ?
 - Ako je objekt izvan slike odnosno okvira video zapisa, njegovo praćenje neće prestati
 - Ako se promijenjeni osvjetljenje između slijednih okvira video zapisa, tada najčešće dolazi do pogrešnog praćenja traženih objekata

Detekcija objekata (pješača, prometnih znakova, vozila)

- Koji algoritmi za praćenje objekata postoje implementirani u OpenCV ?
 - *Median Flow*
 - Izvještava kada praćeni objekt izađe izvan okvira video zapisa
 - Točno prati tražene objekte ako se ne pomiču prebrzo između slijednih okvira video zapisa
 - Odabran u ovom radu za praćenje objekta

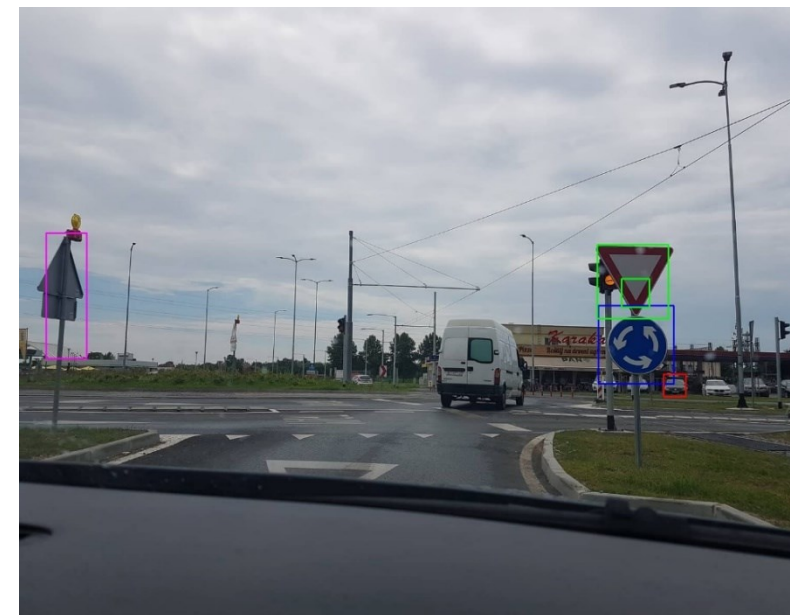
Detekcija objekata (pješaka, prometnih znakova, vozila)

- Rezultati

Naziv detektora	Broj testnih slika	Broj traženih objekata	Broj TP detekcija	Broj FN detekcija	Broj FP detekcija
Prednje i stražnje strane automobila	423	258	233	25	13
Bočne strane automobila	196	81	68	13	0
Prednje i stražnje strane autobusa i kamiona	257	76	61	15	14
Prometnih znakova opasnosti	267	91	85	6	14
Prometnih znakova izričitih naredbi koje karakterizira plava pozadina	234	117	112	5	14
Prometnih znakova izričitih naredbi koje karakterizira crveni rub	288	119	109	10	12
Prometnog znaka obveznog zaustavljanja	230	77	67	10	2
Prometnog znaka raskrižja s cestom koja ima prednost prolaska	313	89	87	2	4
Pješaka	300	100	78	22	16
Semafora	100	240	191	49	9

Detekcija objekata (pješačka, prometnih znakova, vozila)

- Rezultati



P i t a n j a ?

Nadzor vozača (*Driver monitoring*)

Nadzor vozača (*Driver monitoring*)

- Kamera smještena u kontrolnoj ploči vozila
- Potrebno je
 - Detektirati vozačevu glavu u svakom okviru
 - Detektirati ima li vozač spuštenu glavu
 - Detektirati gleda li vozač prema naprijed ili ne
 - Detektirati ima li vozač neuobičajeno dugo zaklopljene oči
- **Rješenje [4] → OpenCV 3.1.0 & C++**

Nadzor vozača (*Driver monitoring*)

- **Viola-Jones algoritam za detekciju objekata**
- Detektori za
 - Prednju stranu lica
 - Profil lica
 - Oči
- Neke od korištenih gotovih OpenCV funkcija
 - *cascadeClassifier()*
 - *detectMultiScale()*
 - *cvtColor()*
 - *rectangle()* ...

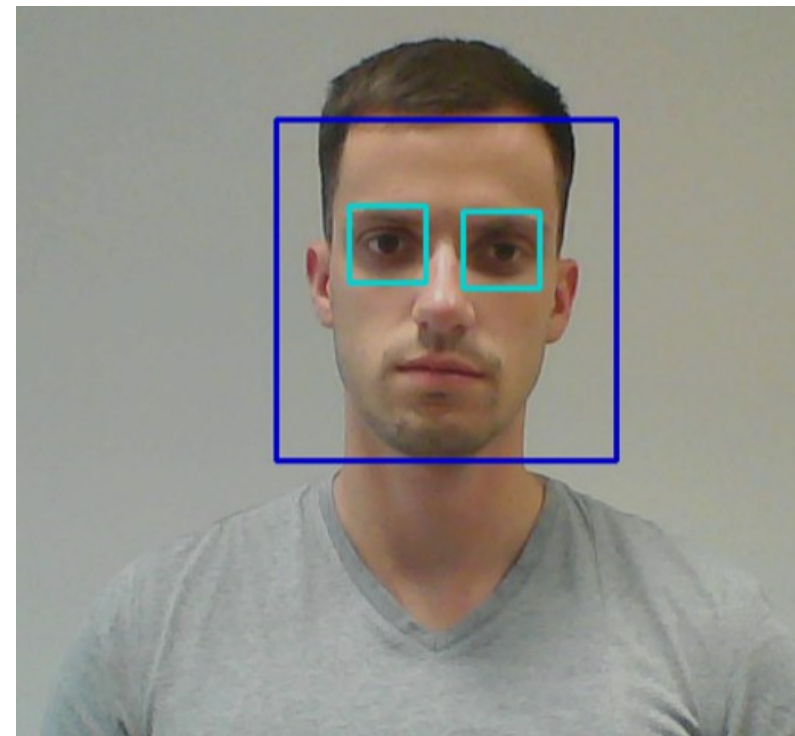
Nadzor vozača (*Driver monitoring*)

- Dijagram toka algoritma za praćenje stanja vozača



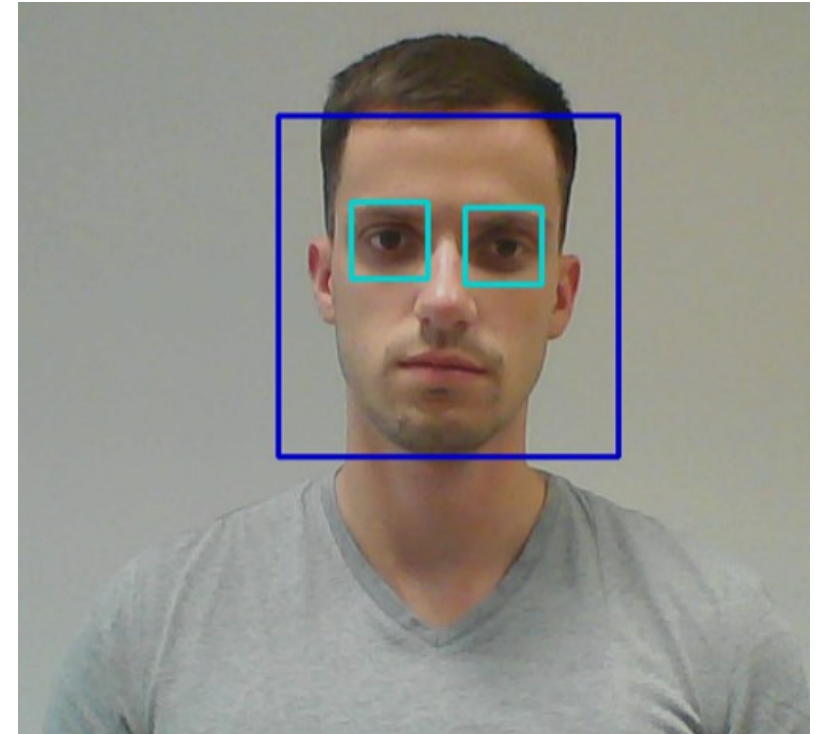
Nadzor vozača (*Driver monitoring*)

- Nakon prvog okvira u kojem se detektira lice, poziva se funkcija koja se izvršava samo jednom
 - Ta funkcija uzima početne podatke o detektiranom licu (veličina glave i koordinate položaja glave)
 - Veličina glave se računa koristeći visinu i širinu pravokutnika koji je označio lice
 - Početne točke (koordinate) kasnije služe za usporedbu prilikom računanja spuštenosti glave i udaljenosti vozačeve glave od kamere



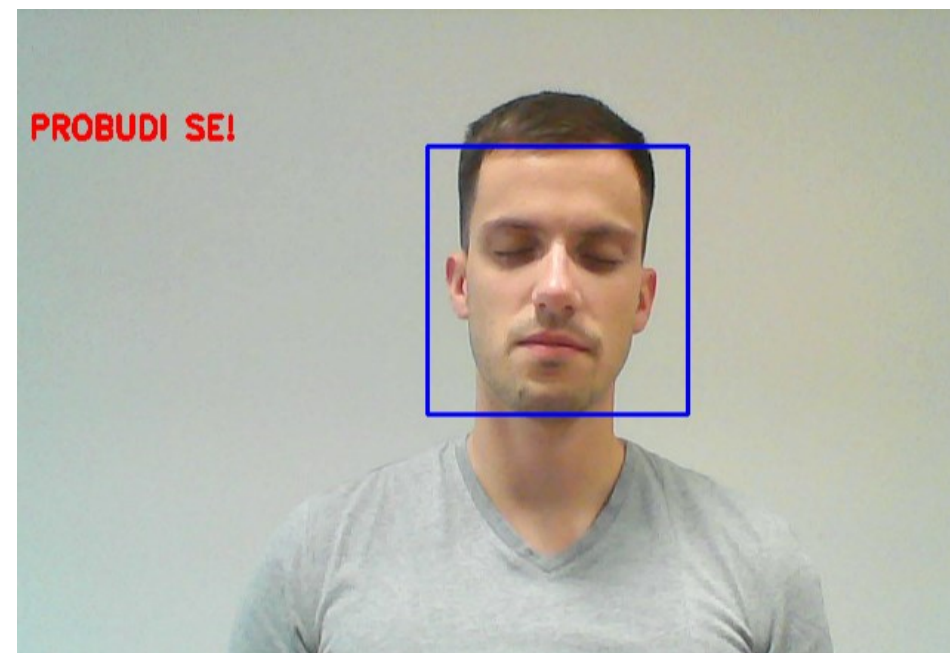
Nadzor vozača (*Driver monitoring*)

- Slučaj više ljudi u kadru? Kako ga riješiti?
 - Algoritam će detektirati najveće lice
- Oči se detektiraju samo ako je prednja strana lica uspješno detektirana
 - Algoritam pretražuje samo dio unutar kojeg se nalazi detektirano lice



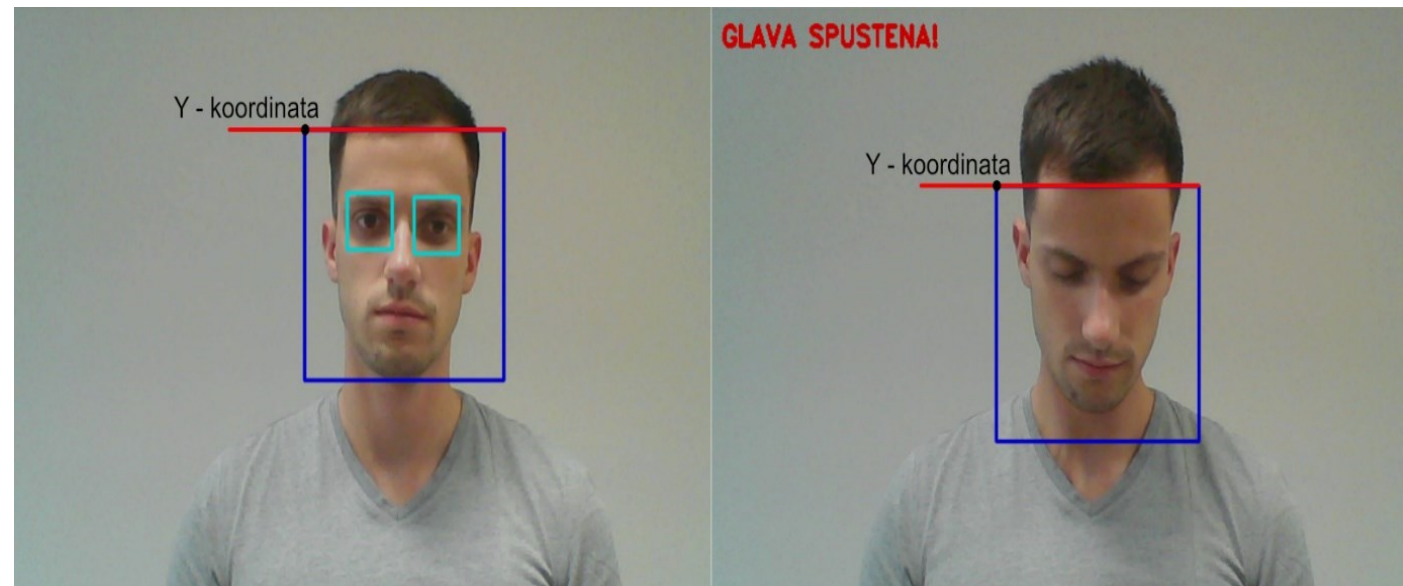
Nadzor vozača (*Driver monitoring*)

- Svaki puta kada je lice detektirano, a oči nisu, poziva se funkcija koja mjeri zatvorenost očiju
- Brojač prati koliko dugo oči nisu detektirane
 - Oglašava se alarm koji ukazuje da vozač najvjerojatnije žmiri



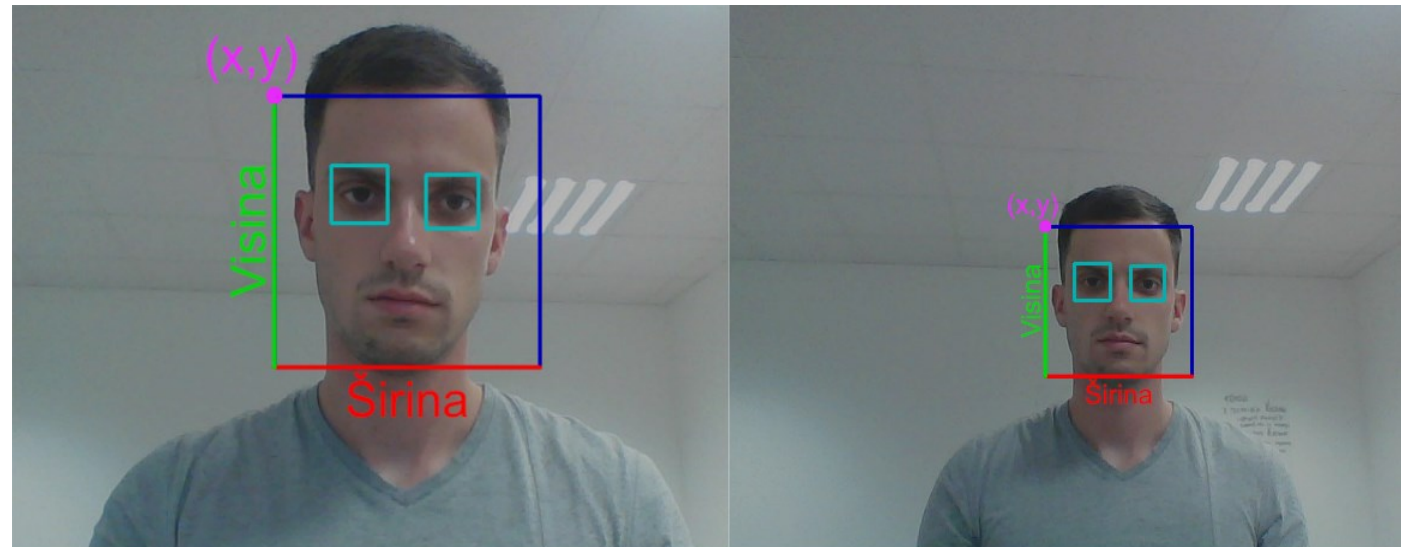
Nadzor vozača (*Driver monitoring*)

- **Sljedeća provjera - provjera spuštenosti glave**
- Provjerava se y koordinata okvira detektiranog lica
- Uspoređuje početnu y koordinatu koja se nalazi u gornjem lijevom kutu, s trenutnom y koordinatom gornjeg lijevog kuta pravokutnika
- Ukoliko vozač spusti glavu prema dolje, y koordinata se mijenja
- Ako vozač glavu drži spuštenu određeni broj okvira, aktivira se alarm i prikazuje se poruka na ekranu



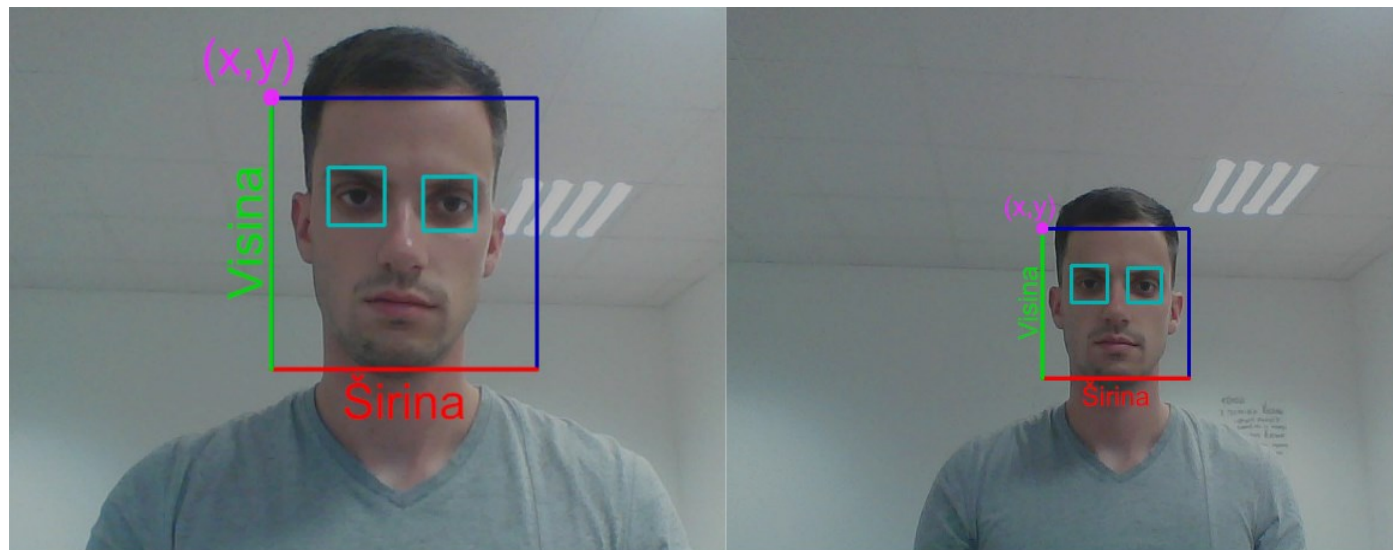
Nadzor vozača (*Driver monitoring*)

- **Sljedeća provjera - provjera položaja glave (približavanje/udaljavanje)**
 - Udaljavanje ili približavanje kameri tijekom vožnje – opet se promatra y koordinata koja se mijenja
 - Mjeri se i površina pravokutnika koji označava lice
- Ovakvom promjenom položaja i udaljenosti od kamere dobili bi se krivi rezultati prilikom provjere spuštenosti glave
- Sustav bi detektirao spuštenost glave, a zapravo se vozač samo udaljio od kamere



Nadzor vozača (*Driver monitoring*)

- Kako bi se izbjegla ovakva greška, radi se dodatna provjera koja uspoređuje početnu površinu s trenutnom
- Ukoliko je površina veća ili manja od određenog praga usporedno s početnom, površina glave i y koordinate se ažuriraju na trenutnu veličinu i položaj



Nadzor vozača (*Driver monitoring*)

- **Sljedeća provjera – detekcija i provjera profila lica (okret glave u stranu)**
 - Ako algoritam ne može detektirati prednju stranu lica, pokušat će detektirati profil lica
 - Kada se detektira profil lica, pokreće se brojač i broji koliko dugo vozač gleda u stranu
- Aktivira se alarm za upozorenje koji govori vozaču da vrati pogled prema naprijed
- Kako bi se smanjilo korištenje više nepotrebnih detektora (lijevi i desni profil)
 - Ukoliko nije detektiran lijevi profil, pozvat će funkciju koja će okrenuti sliku oko y osi, odnosno postići će se efekt zrcala



P i t a n j a ?

Kamera kao zamjena za zrcalo
(Camera mirror replacement)

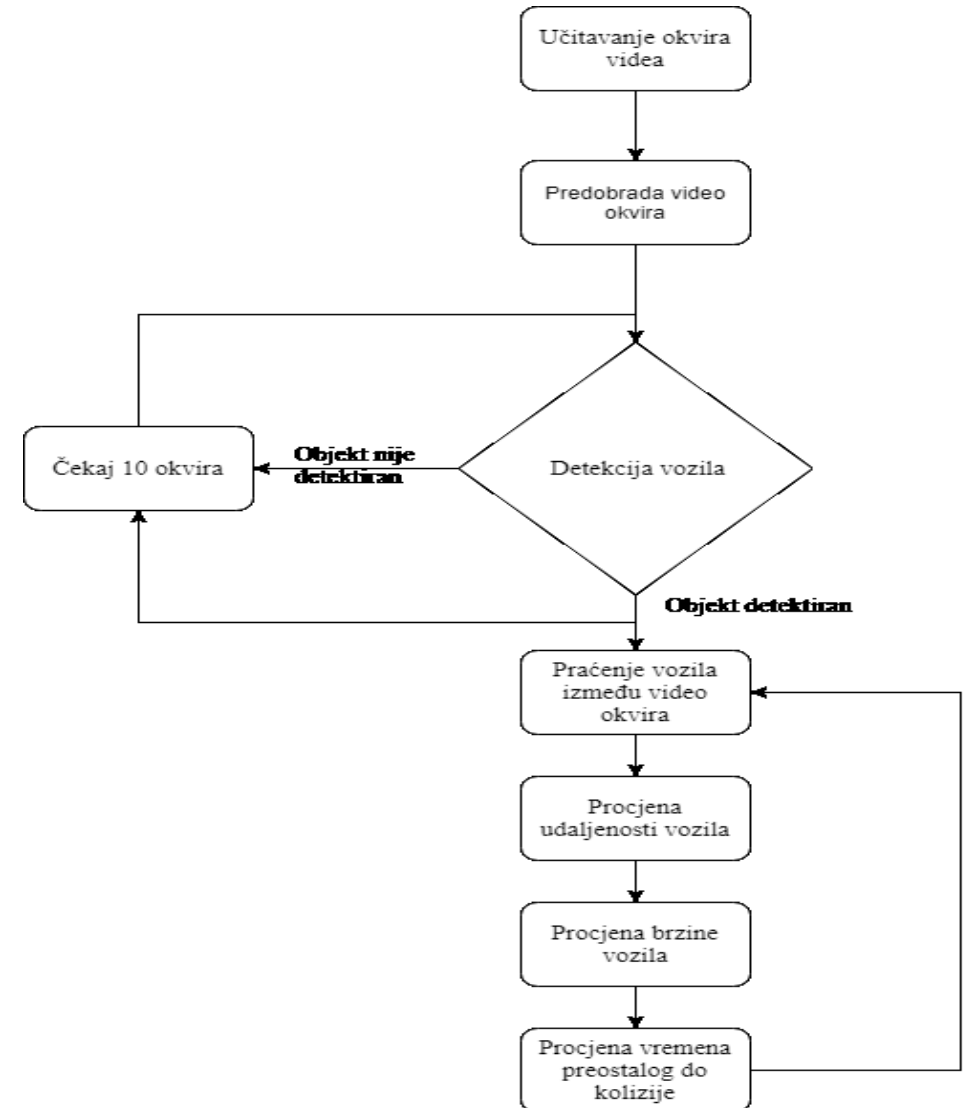
Kamera kao zamjena za zrcalo

- Kamera smještena u lijevom bočnom vozačevom retrovizoru i snima prostor iza vozila
- Potrebno je
 - Detektirati vozilo koje dolazi odostraga
 - Procijeniti njegovu udaljenost
 - Procijeniti njegovu brzinu
 - Procijeniti vrijeme preostalo do potencijalne kolizije (tj. koliko je preostalo vremena za sigurnu promjenu vozne trake - prestrojavanje)
- **Rješenje [5] → Python 3.6.5 & OpenCV 4.0.0**



Kamera kao zamjena za zrcalo

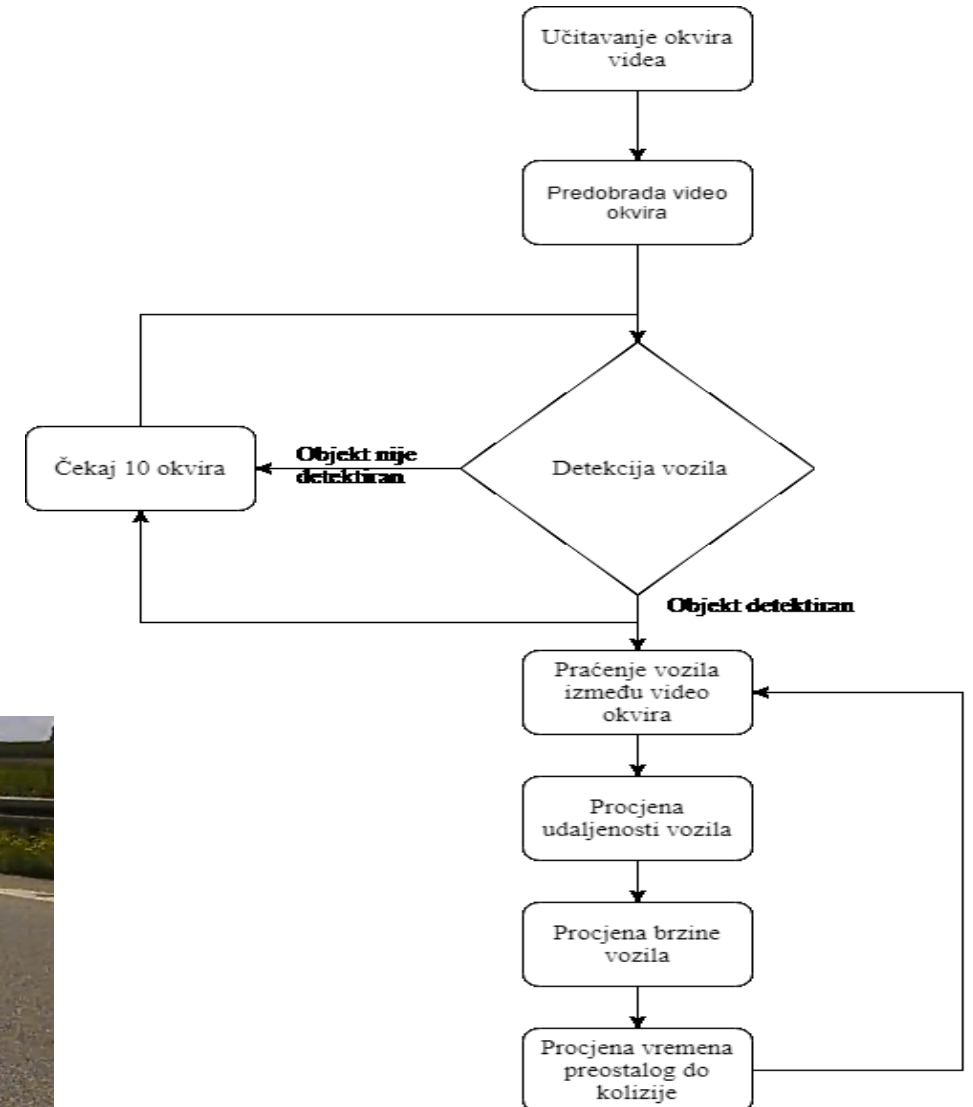
- Algoritam podijeljen u 4 dijela
 - Predobrada video okvira
 - Detekcija nadolazećeg vozila
 - Praćenje detektiranog vozila između uzastopnih video okvira
 - Procjena brzine, udaljenosti i vremena preostalog do kolizije



Kamera kao zamjena za zrcalo

- Predobrada

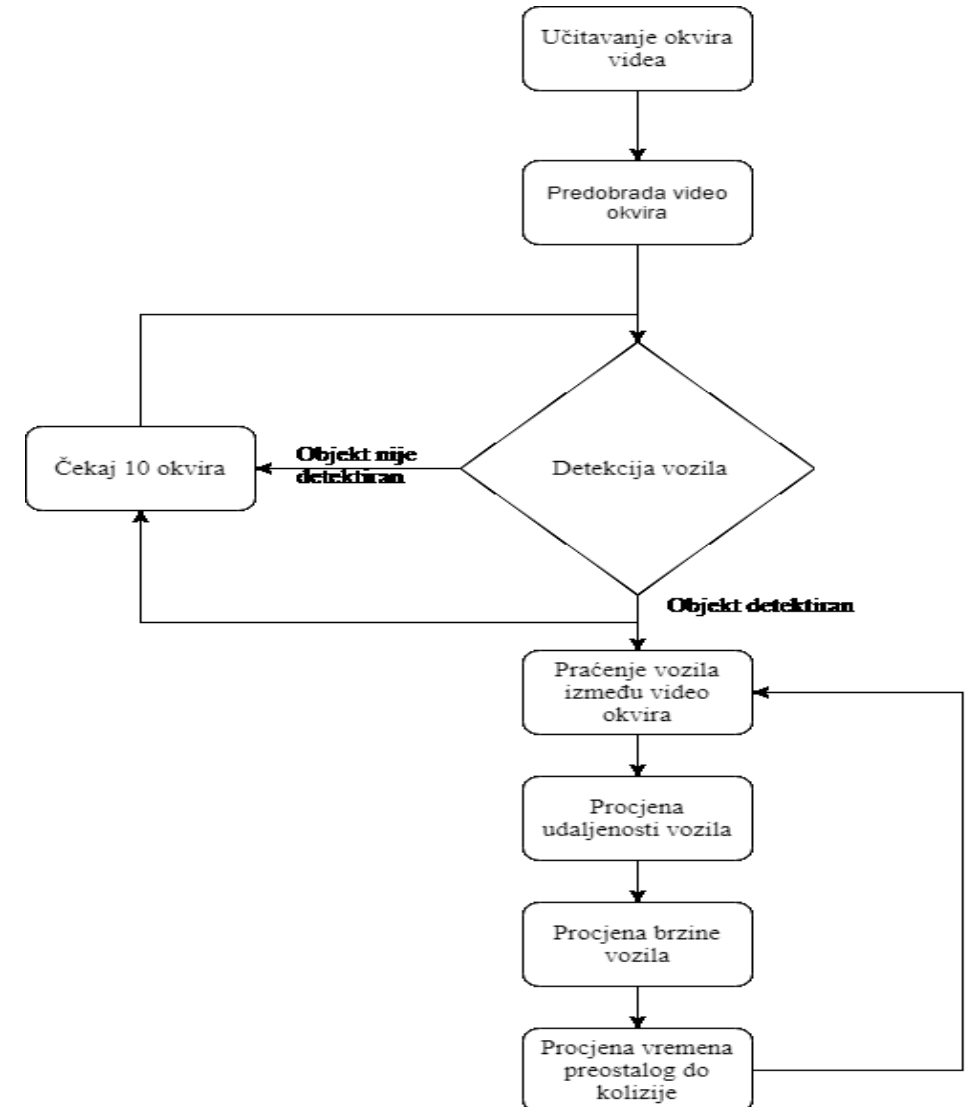
- Izdvaja se dio okvira koji je od interesa (ROI), ostatak se zanemaruje
- Npr. često se u video okvirima nalazi nebo ili okolina ceste → izbacuje se
- Smanjuje se mogućnost pogreške i povećava se brzina rada



Kamera kao zamjena za zrcalo

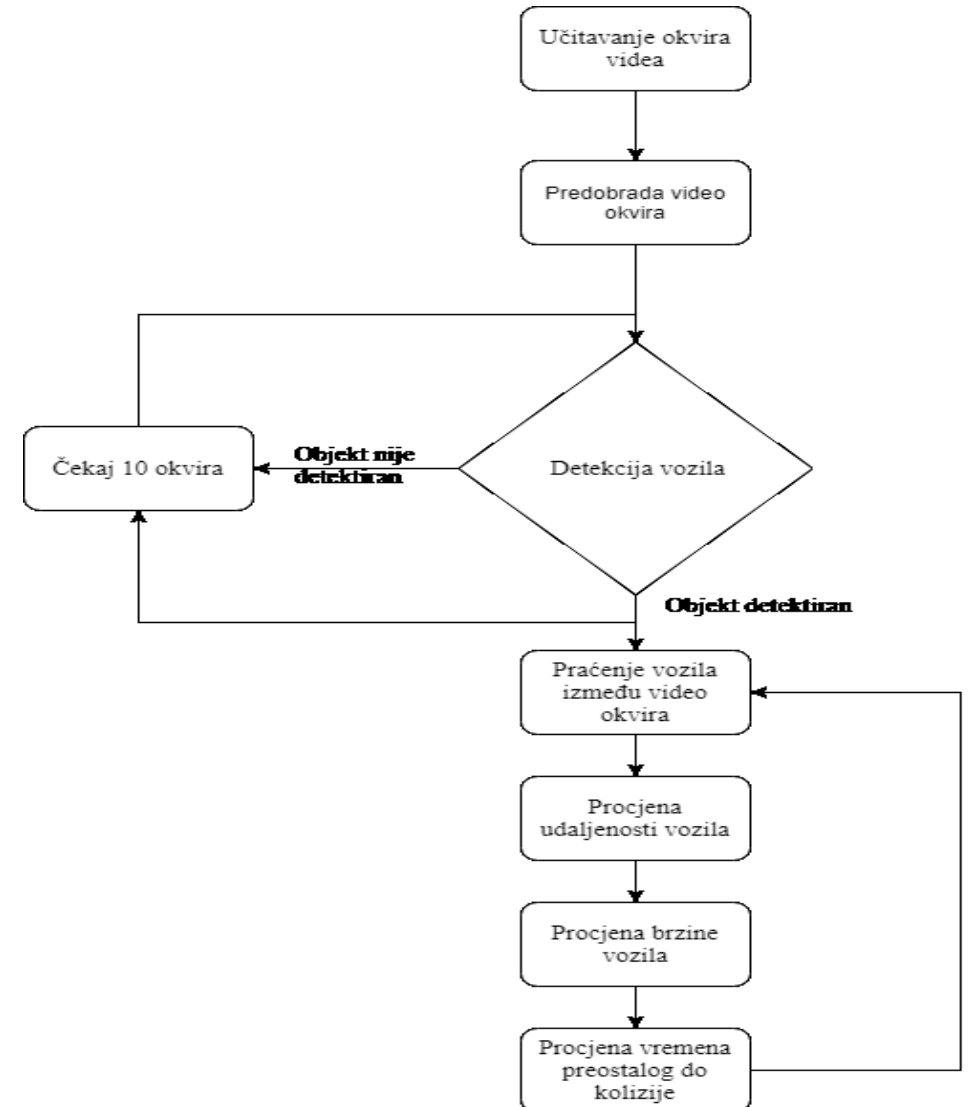
- Detekcija nadolazećeg vozila

- Dobiva se informacija gdje se u video okviru nalazi vozilo
- Viola-Jones algoritam (treniranje detektora)



Kamera kao zamjena za zrcalo

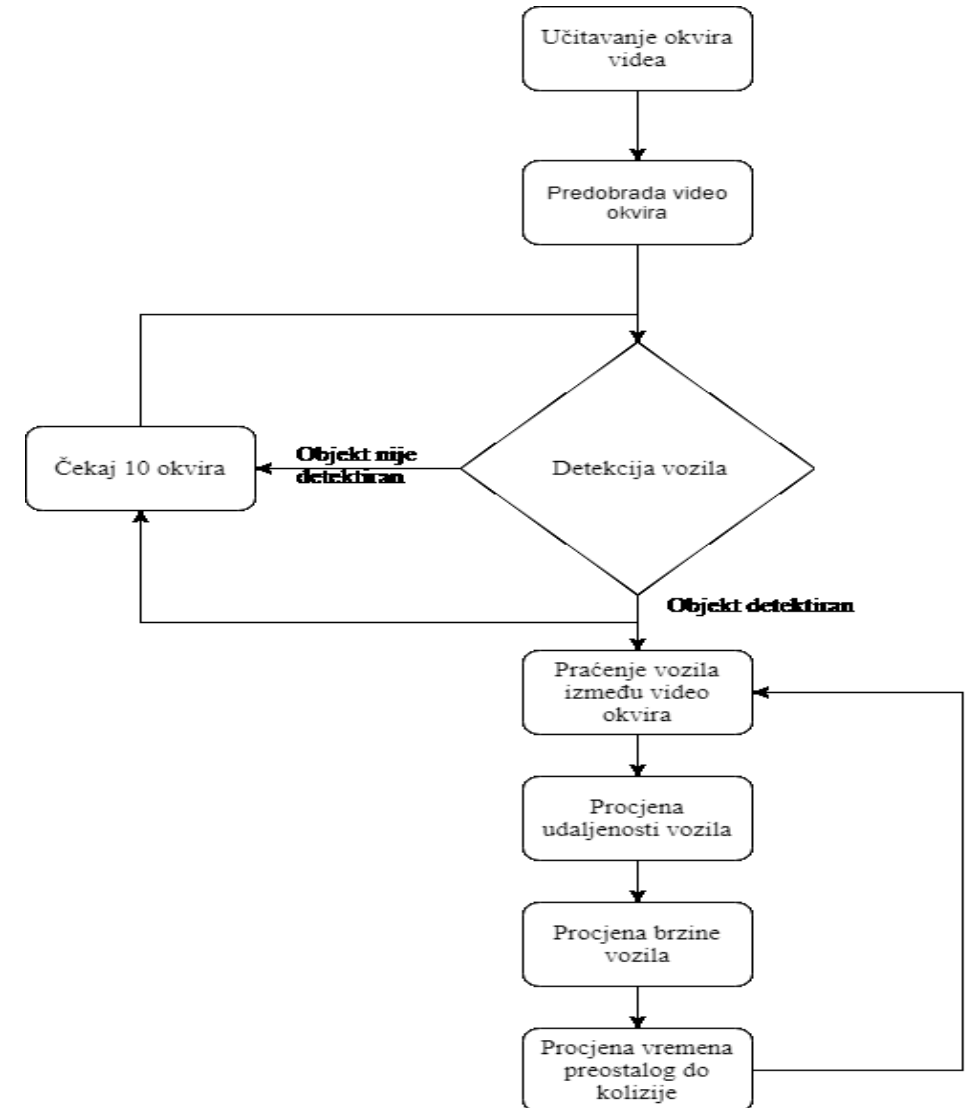
- Praćenje detektiranog vozila između uzastopnih video okvira
 - Omogućuje povećanje brzine obrade video okvira jer samo praćenje zahtjeva manje procesorskog vremena nego detekcija
 - Praćenjem detektiranog objekta se ne onemogućuje detekcija, nego se na svakom 10. video okviru vrši ponovna detekcija
 - Ponovnom detekcijom svakih 10 video okvira osigurava se detekcija novih objekata od interesa, ako su ušli u video okvir
 - Za praćenje objekata između video okvira koristi se *Median Flow* algoritam koji je dio OpenCV biblioteke



Kamera kao zamjena za zrcalo

- Procjena brzine, udaljenosti i vremena preostalog do kolizije

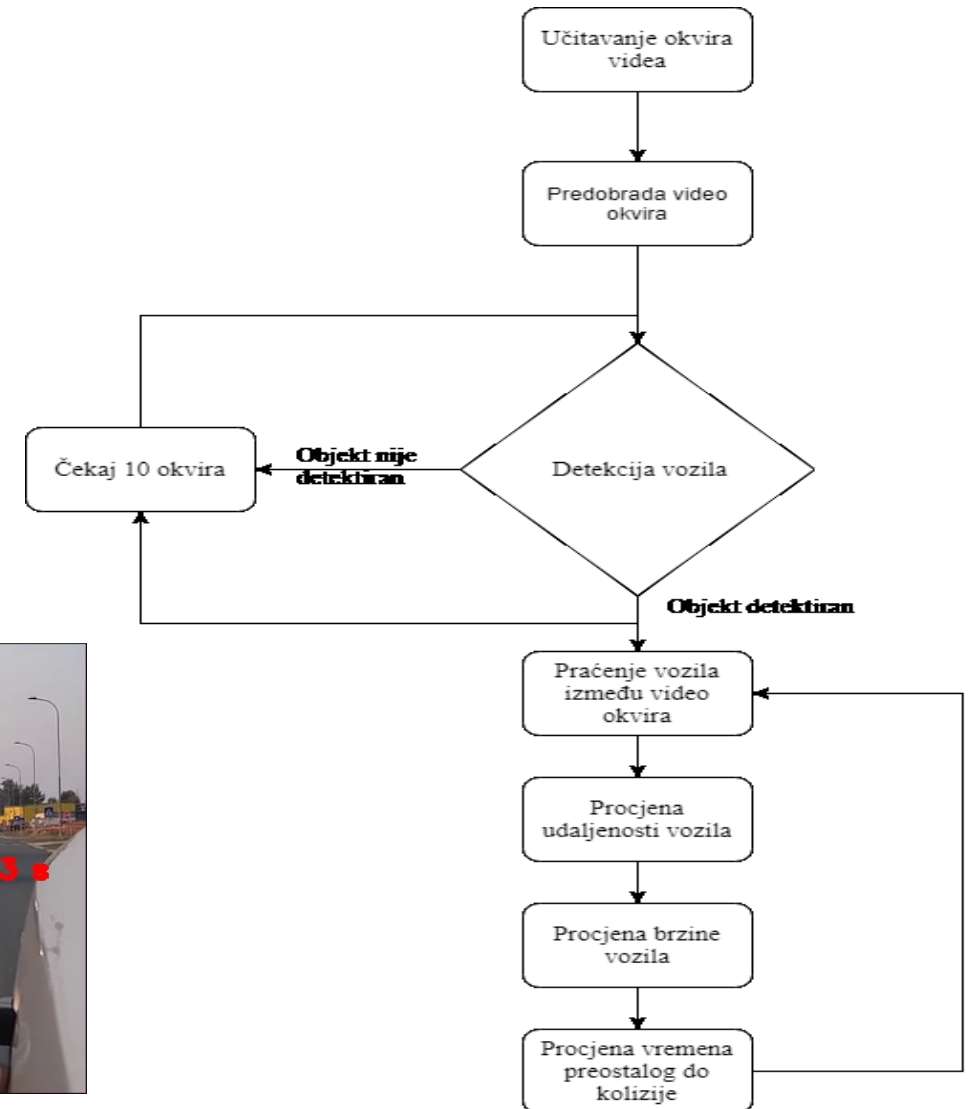
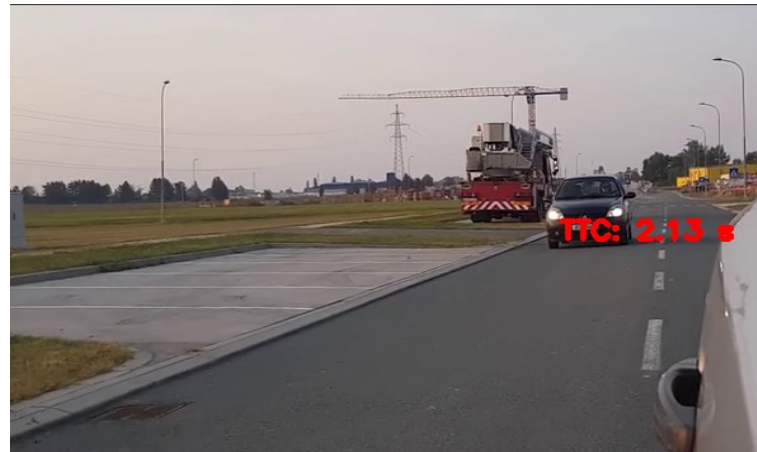
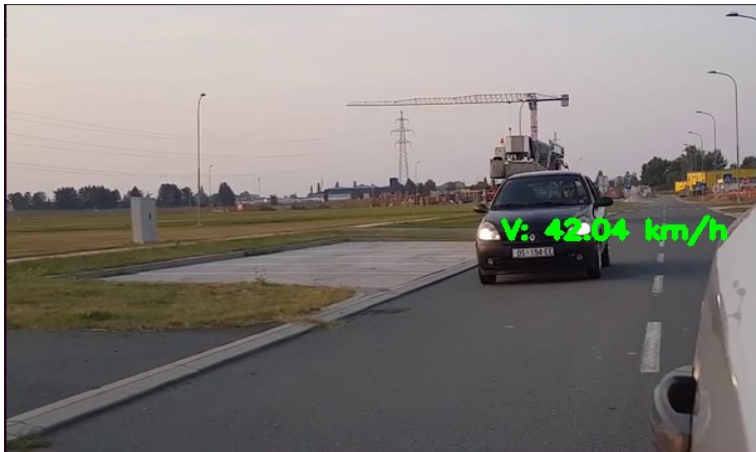
- Dobivaju se informacije na temelju kojih vozač može odlučiti može li sigurno promijeniti vožnju
- Prvo se procjenjuje udaljenost detektiranog vozila (potrebno poznavati parametre kamere – širinu senzora i žarišnu duljinu)



Kamera kao zamjena za zrcalo

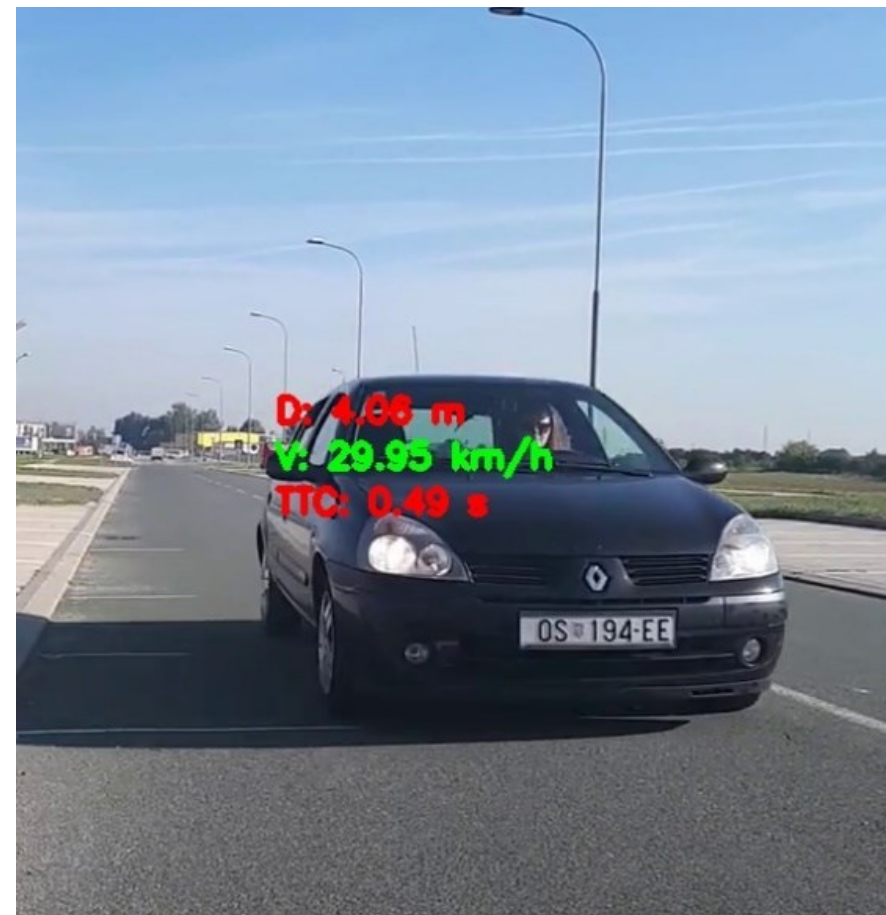
- Procjena brzine, udaljenosti i vremena preostalog do kolizije

- Nakon toga se procjenjuju brzina nadolazećeg vozila te vrijeme preostalo do kolizije



Kamera kao zamjena za zrcalo

- Rezultati



P i t a n j a ?

**Detekcija i klasifikacija parkirnog
mjesta i pomoć pri parkiranju**

Detekcija i klasifikacija parkirnog mjesta

- Kamera smještena u lijevom i desnom bočnom retrovizoru i snima prostor bočno od vozila (rješenje razvijeno u simulatoru)
- Potrebno je
 - Detektirati parkirna mjesta s obje strane ceste
 - Klasificirati parkirna mjesta kao zauzeta/slobodna
- Nakon toga je potrebno uparkirati vozilo na detektirano slobodno parkirno mjesto
 - Vožnjom prema naprijed
 - Vožnjom prema nazad
 - Za obje strane ceste (nije isti položaj vozila u odnosu na detektirani parking)



Detekcija i klasifikacija parkirnog mjesta

- **Rješenje [6]**

- Algoritam za detekciju parkinga je napisan u **Python** programskom jeziku, uz korištenje **OpenCV** i **NumPy** biblioteka
- Algoritam je razvijan na Ubuntu 16.04 LTS operacijskom sustavu
- Prilikom razvoja i testiranja korišten je **CARLA Simulator** verzije 0.9.6

Detekcija i klasifikacija parkirnog mjesta

Prikaz okvira dobivenog s kamere postavljene na lijevoj strani vozila

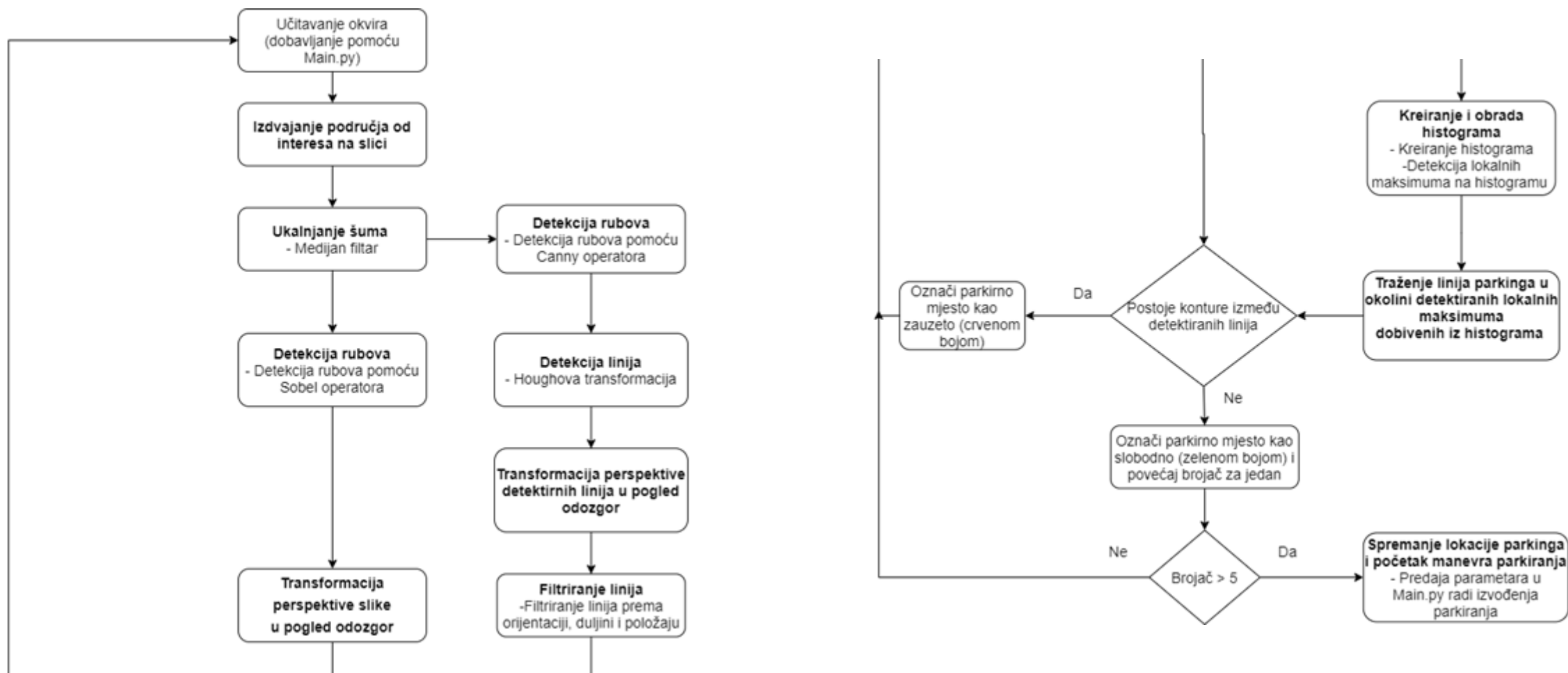


Prikaz okvira dobivenog s kamere postavljene na desnoj strani vozila



Detekcija i klasifikacija parkirnog mjesta

Dijagram toka algoritma za detekciju i klasifikaciju parkirnog mjesta



Detekcija i klasifikacija parkirnog mjesta

Ulazni okvir



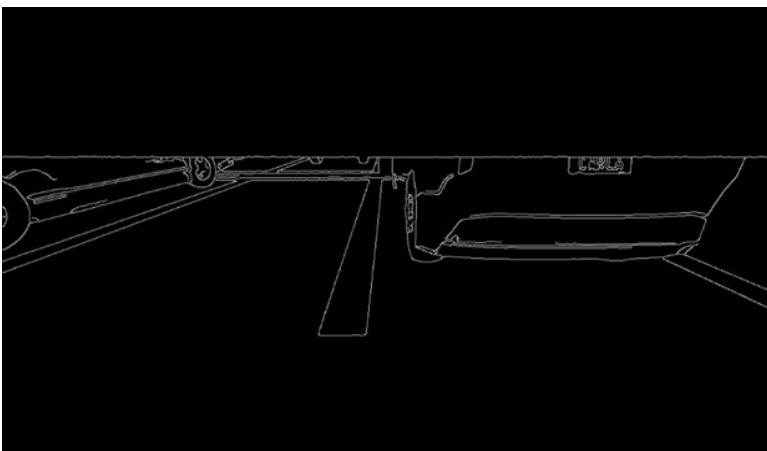
Okvir nakon izdvajanja ROI



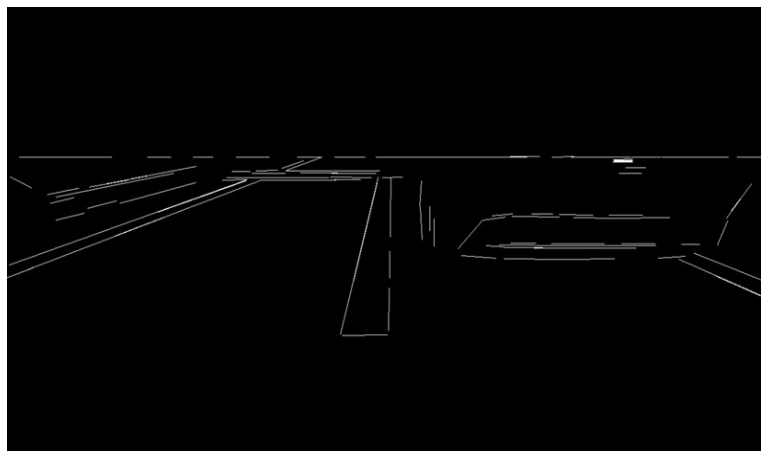
Roi + Median filter



Canny detekcija rubova



Hough-ova transformacija

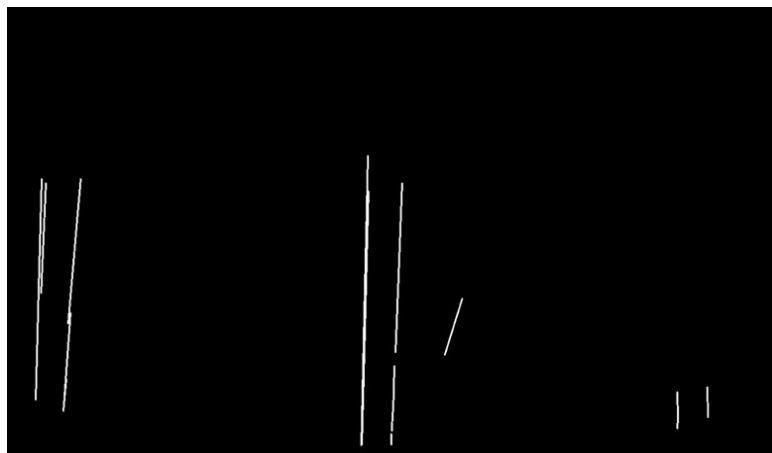


Transformacija perspektive

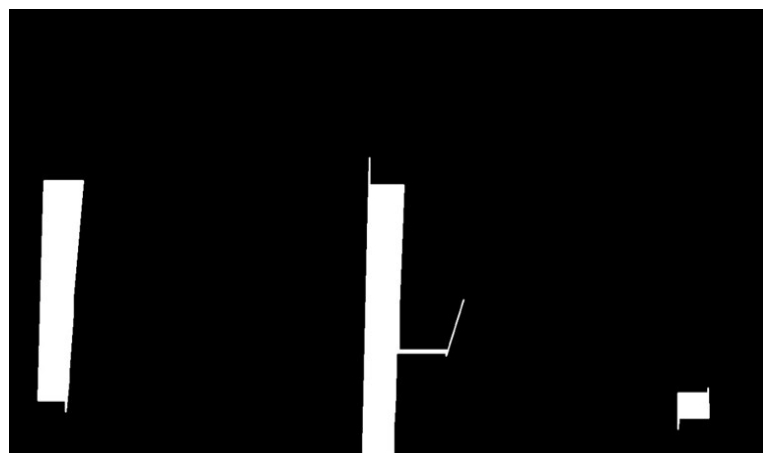


Detekcija i klasifikacija parkirnog mjesta

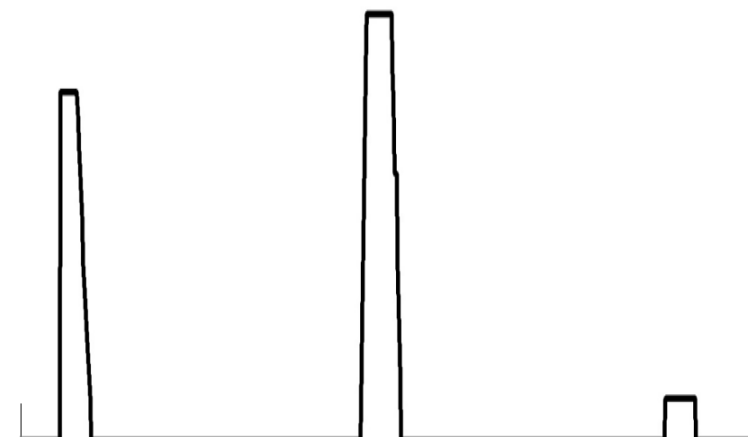
Filtriranje linija



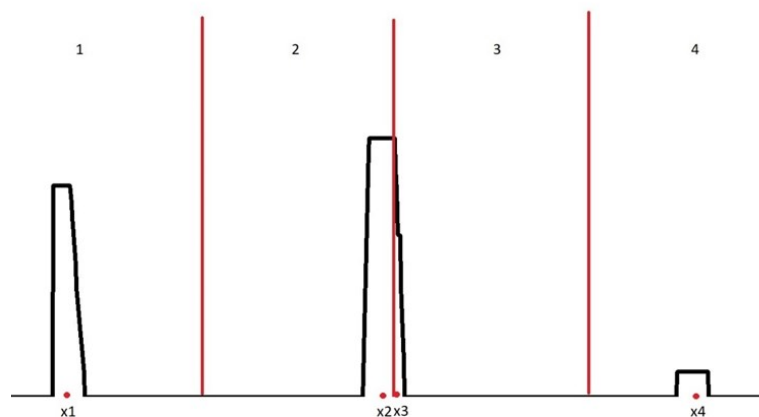
Morfološko filtriranje zatvaranjem



Histogram nakon morf. filtriranja

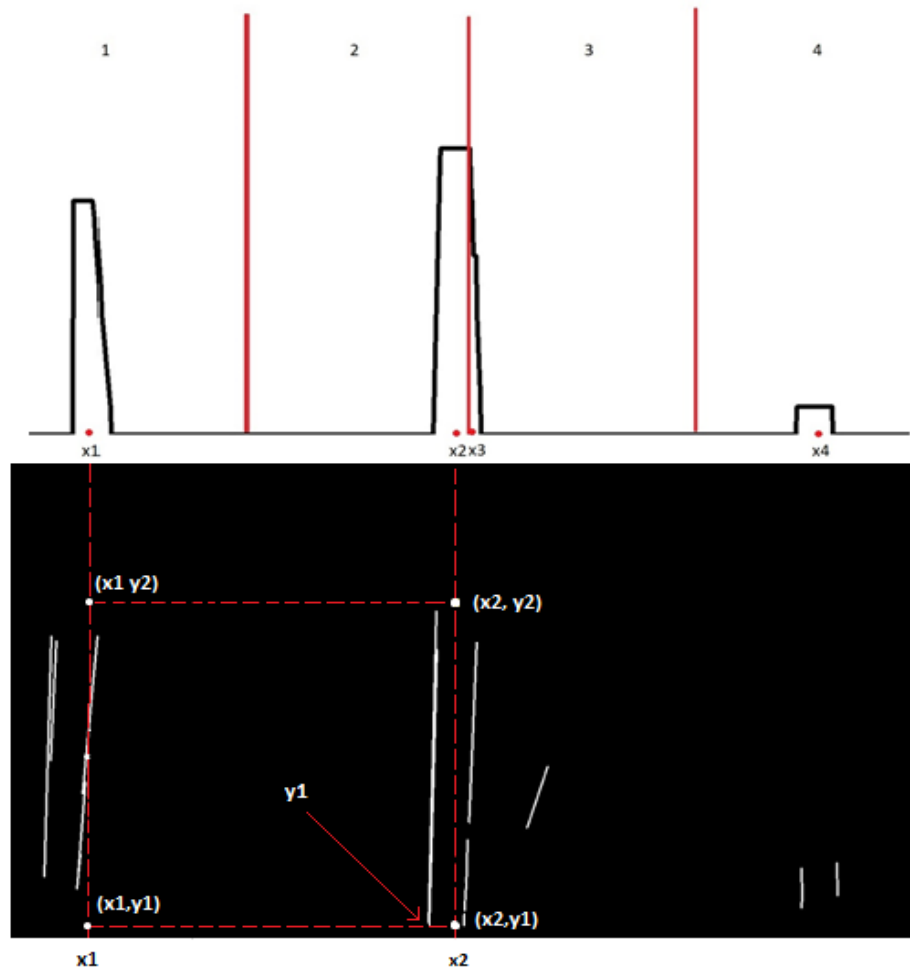


Detekcija maksimuma na histogramu



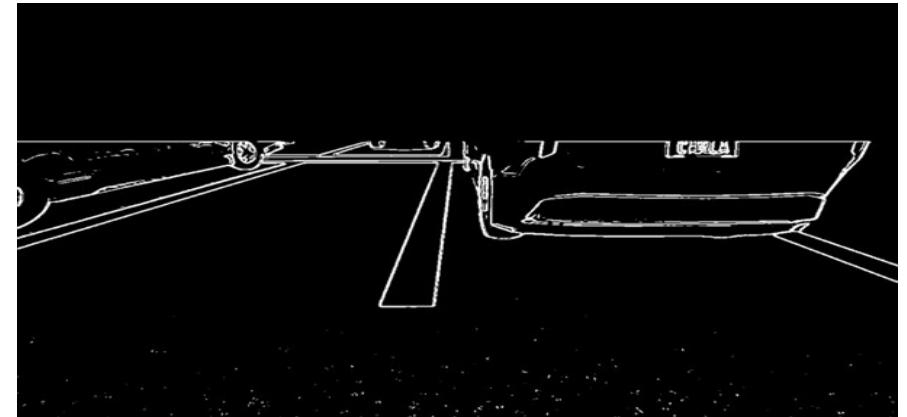
Detekcija i klasifikacija parkirnog mjesta

- Određivanje koordinata parkinga iz maksimuma na histogramu

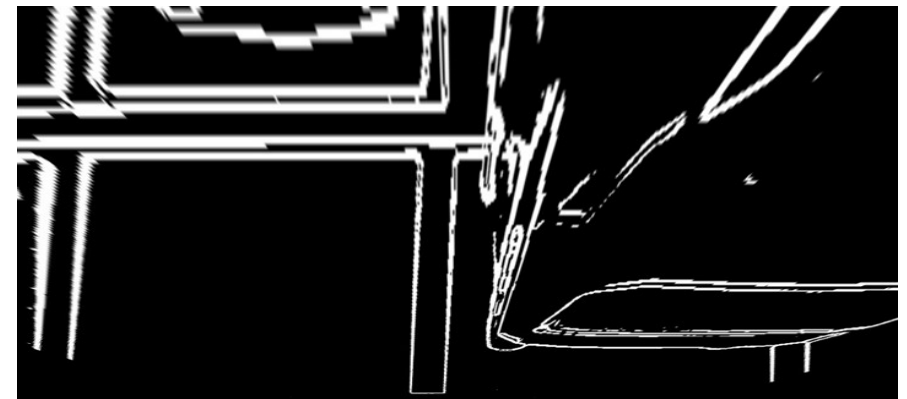


Klasifikacija detektiranog parkirnog mjesta

- Sobel detekcija rubova

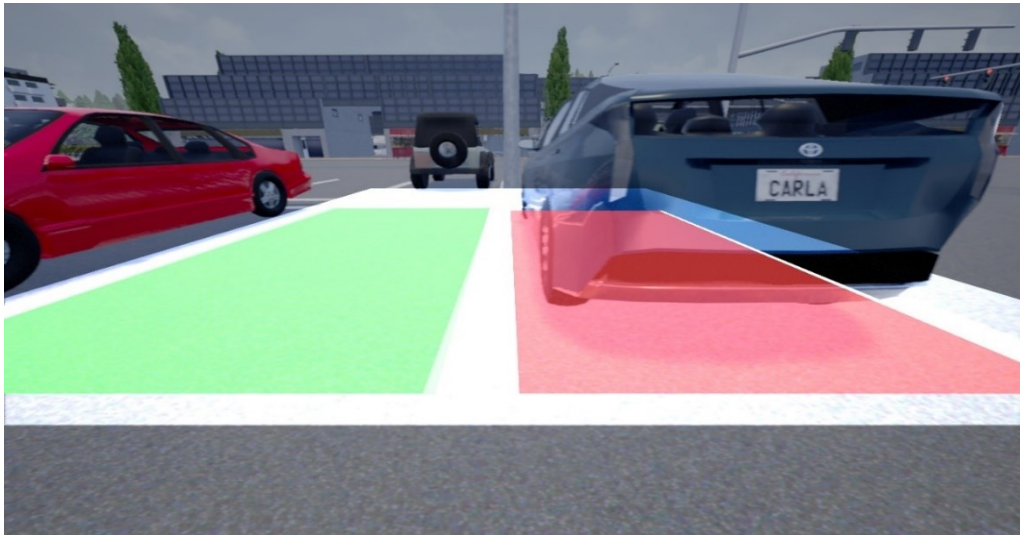


- Transformacija perspektive i klasifikacija

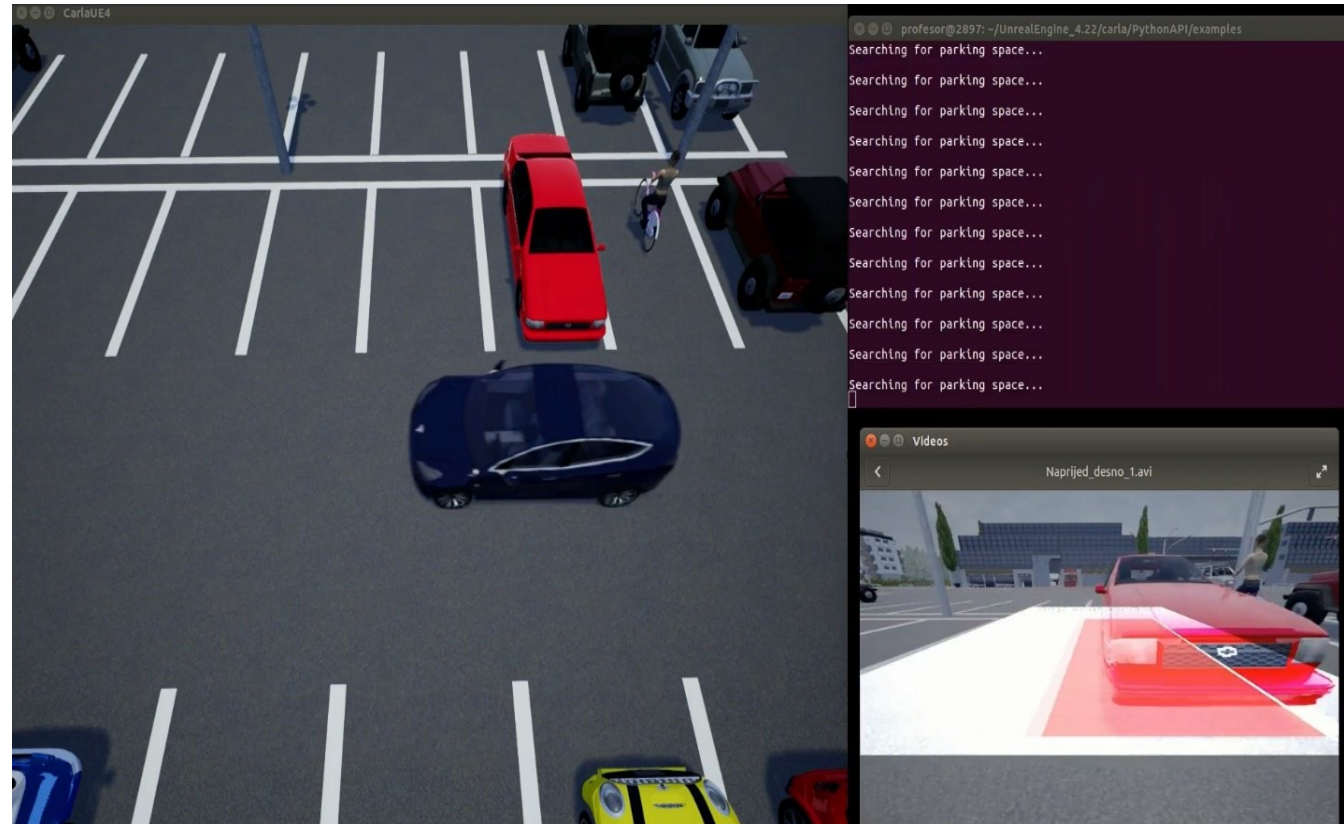


Detekcija i klasifikacija parkirnog mjesta

Izlazni okvir

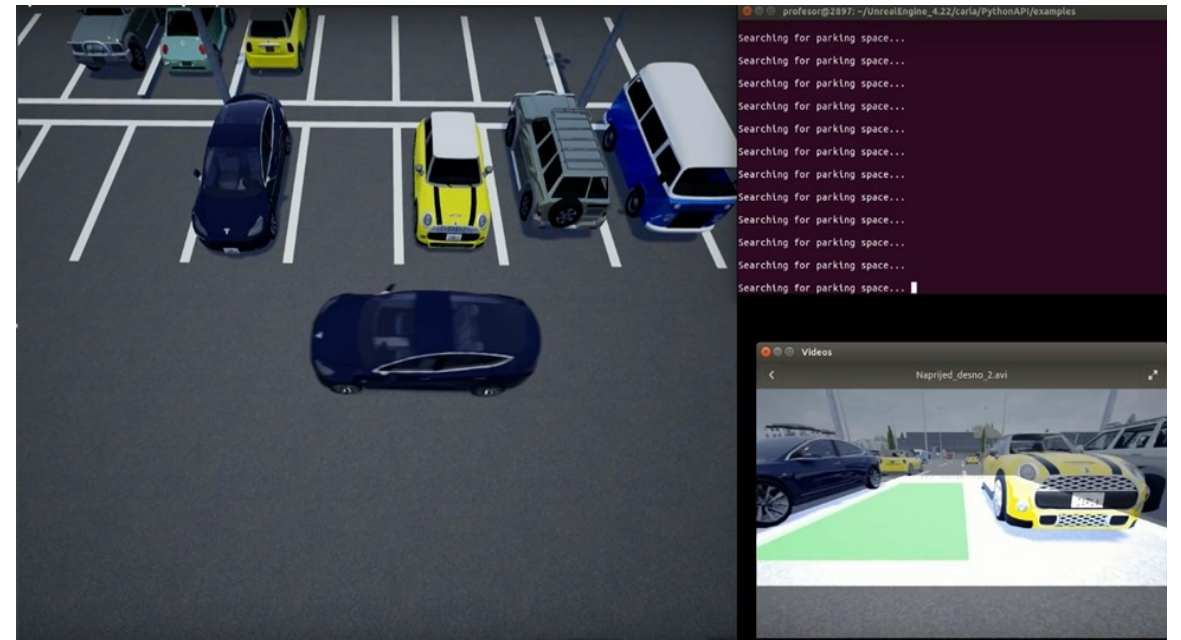


Prikaz detekcije parkinga u stvarnom vremenu



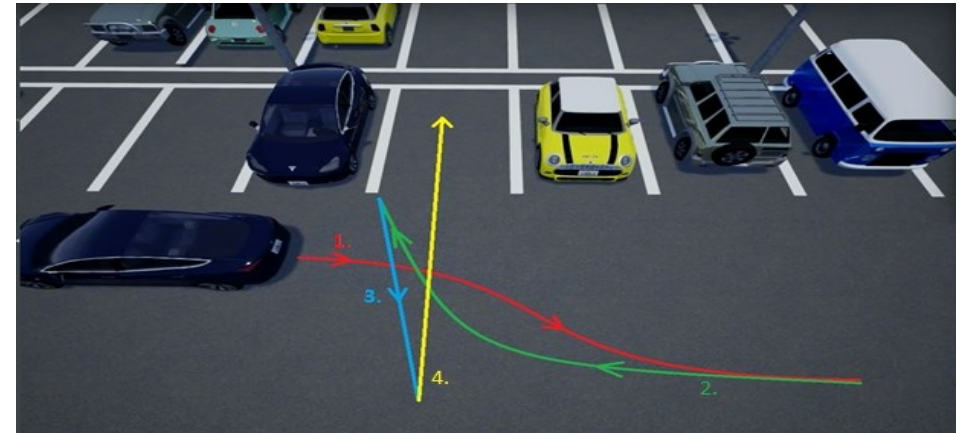
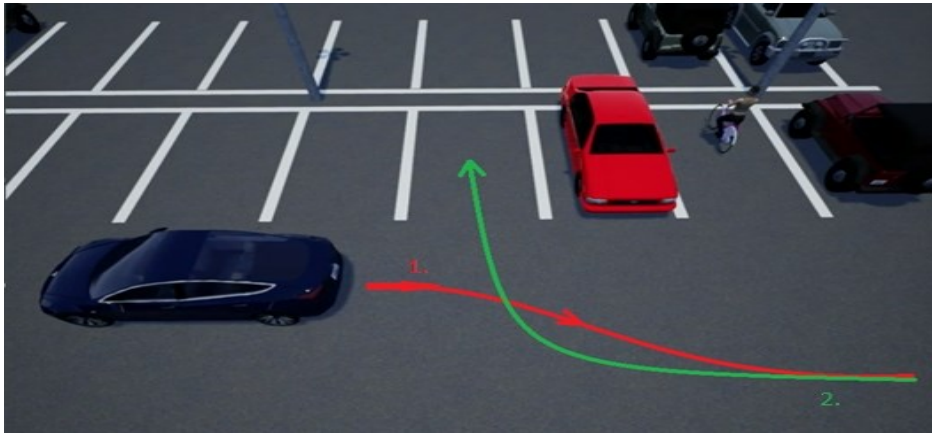
Detekcija i klasifikacija parkirnog mjesta

Prikaz detekcije parkinga u stvarnom vremenu



Detekcija i klasifikacija parkirnog mjesta

Manevriranje vozilom



P i t a n j a ?

Korisna literatura

- [1] V. Appia et al., „Surround view camera system for ADAS on TI's TDAx SoCs", PDF, Texas Instruments
- [2] „Surround View and Front Camera Analytics on TDA2x SoC", VIDEO
 - http://www.ti.com/general/docs/video/watch.tsp?entryid=0_t71u92t9#
- [3] Ciberlin, Juraj; Grbić, Ratko; Teslić, Nikola; Pilipović, Miloš, „Object detection and object tracking in front of the vehicle using front view camera", 2019 Zooming Innovation in Consumer Technologies Conference (ZINC), Novi Sad, Srbija, 2019. str. 27-32
- [4] Mašanović, Luka; Vranješ, Mario; Džakula, Radomir; Lukač, Željko, „Driver monitoring using the in-vehicle camera", 2019 Zooming Innovation in Consumer Electronics International Conference, Novi Sad, 2019. str. 1-6
- [5] Ćosić, Luka; Vranješ, Mario; Ilkić, Veljko; Mihić, Velibor, „Time to collision estimation for vehicles coming from behind using in-vehicle camera", 2019 Zooming Innovation in Consumer Electronics International Conference, Novi Sad, 2019., str. 1-4
- [6] M. Kuprešak, M. Vranješ „Solution for Autonomous Vehicle Parking", FERIT Osijek, 2019
- [7] D. Špoljar, M. Vranješ: „Lane Detection and Lane Departure Warning Using Front View Camera in Vehicle", FERIT, 2019